JARROD BESNER · AGENTIC

Python source book

All Python used by Agent F (supervised Cursor chat) and Agent H (autonomous Agentic bot). Live place_* is Robinhood MCP, not a side effect of this code. H's standing prompt is markdown: playbooks/agent_h_autonomous.PROMPT.md · not included here.

22 files · generated 2026-09-02 08:37 UTC
Language: Python 3 · Letter portrait

Contents

```
1  """Phase 2 read-only signal pipeline (Agents A·E + I)."""
2
3  __version__ = "0.2.0"
```

```python
from __future__ import annotations

import json
from datetime import datetime, timezone
from pathlib import Path
from typing import Any


ROOT = Path(__file__).resolve().parents[1]
SIGNALS = ROOT / "signals"
JOURNAL = ROOT / "journal"
DATA_RAW = ROOT / "data" / "raw"
CONFIG = ROOT / "config" / "rules.json"


def utc_now_iso() -> str:
    return datetime.now(timezone.utc).replace(microsecond=0).isoformat()


def load_rules() -> dict[str, Any]:
    return json.loads(CONFIG.read_text(encoding="utf-8"))


def write_json(path: Path, payload: Any) -> Path:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(payload, indent=2, sort_keys=False) + "\n", encoding="utf-8")
    return path


def append_jsonl(path: Path, row: dict[str, Any]) -> Path:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("a", encoding="utf-8") as f:
        f.write(json.dumps(row, sort_keys=False) + "\n")
    return path


def read_json(path: Path) -> Any:
    return json.loads(path.read_text(encoding="utf-8"))
```

```python
from __future__ import annotations

from datetime import datetime, time
from zoneinfo import ZoneInfo
from typing import Any

ET = ZoneInfo("America/New_York")
RTH_START = time(9, 30)
RTH_END = time(16, 0)
NO_NEW_ENTRIES_AFTER = time(15, 45)


def now_et(now: datetime | None = None) -> datetime:
    if now is None:
        return datetime.now(ET)
    if now.tzinfo is None:
        return now.replace(tzinfo=ET)
    return now.astimezone(ET)


def is_weekday(now: datetime | None = None) -> bool:
    return now_et(now).weekday() < 5


def is_rth(now: datetime | None = None) -> bool:
    """Mon·Fri 09:30 inclusive through 16:00 exclusive America/New_York."""
    dt = now_et(now)
    if not is_weekday(dt):
        return False
    t = dt.time()
    return RTH_START <= t < RTH_END


def entries_open(now: datetime | None = None) -> bool:
    """New entries only in RTH before 15:45 ET."""
    dt = now_et(now)
    if not is_rth(dt):
        return False
    return dt.time() < NO_NEW_ENTRIES_AFTER


def flatten_window(now: datetime | None = None) -> bool:
    """Still RTH, but new entries are closed (15:45·16:00 ET)."""
    dt = now_et(now)
    if not is_rth(dt):
        return False
    return dt.time() >= NO_NEW_ENTRIES_AFTER


def session_gate(now: datetime | None = None) -> dict[str, Any]:
    dt = now_et(now)
    rth = is_rth(dt)
    open_for_entry = entries_open(dt)
    reason = None
    if not rth:
        reason = "outside_rth"
    elif not open_for_entry:
        reason = "no_new_entries_after_1545"
    return {
        "timezone": "America/New_York",
        "now_et": dt.isoformat(),
        "is_rth": rth,
        "entries_open": open_for_entry,
        "flatten_window": flatten_window(dt),
        "reason": reason,
    }
```

```python
from __future__ import annotations

from typing import Any, Iterable

# Robinhood MCP has no `open=true` filter. Working tickets are these states.
OPTION_WORKING_STATES = frozenset(
    {"queued", "confirmed", "partially_filled", "pending_cancelled"}
)
EQUITY_WORKING_STATES = frozenset(
    {"new", "queued", "confirmed", "unconfirmed", "partially_filled"}
)


def normalize_state(value: Any) -> str:
    return str(value or "").strip().lower()


def is_working_option_state(state: Any) -> bool:
    return normalize_state(state) in OPTION_WORKING_STATES


def is_working_equity_state(state: Any) -> bool:
    return normalize_state(state) in EQUITY_WORKING_STATES


def working_orders(
    option_orders: Iterable[dict[str, Any]] | None = None,
    equity_orders: Iterable[dict[str, Any]] | None = None,
) -> list[dict[str, Any]]:
    found: list[dict[str, Any]] = []
    for row in option_orders or []:
        if is_working_option_state(row.get("state") or row.get("status")):
            found.append({"asset_class": "option", **row})
    for row in equity_orders or []:
        if is_working_equity_state(row.get("state") or row.get("status")):
            found.append({"asset_class": "equity", **row})
    return found


def has_working_orders(
    option_orders: Iterable[dict[str, Any]] | None = None,
    equity_orders: Iterable[dict[str, Any]] | None = None,
) -> bool:
    return bool(working_orders(option_orders, equity_orders))
```

```python
from __future__ import annotations

from typing import Any


CRYPTO_OBJECT_TYPES = {"currency_pair", "tokenized_stock"}
EQUITY_OBJECT_TYPES = {"instrument"}
INDEX_OBJECT_TYPES = {"index"}
OPTION_OBJECT_TYPES = {"option_strategy", "option"}


def _normalize_symbol(value: str | None) -> str | None:
    if not value:
        return None
    return value.strip().upper()


def extract_watchlist_symbols(
    watchlists: list[dict[str, Any]],
    items_by_list: dict[str, list[dict[str, Any]]],
    option_watchlist_items: list[dict[str, Any]] | None,
    *,
    include_crypto: bool = False,
    include_options_watchlist: bool = True,
) -> dict[str, Any]:
    """Agent A: union symbols from all lists; exclude crypto unless enabled."""
    equities: dict[str, dict[str, Any]] = {}
    indexes: dict[str, dict[str, Any]] = {}
    skipped_crypto: list[str] = []
    sources: dict[str, list[str]] = {}

    for wl in watchlists:
        list_id = wl.get("id")
        name = wl.get("display_name") or list_id
        for item in items_by_list.get(list_id, []):
            obj_type = (item.get("object_type") or item.get("type") or "").lower()
            symbol = _normalize_symbol(
                item.get("symbol")
                or item.get("display_symbol")
                or item.get("equity_symbol")
                or item.get("chain_symbol")
            )
            if not symbol:
                continue
            sources.setdefault(symbol, [])
            if name not in sources[symbol]:
                sources[symbol].append(str(name))

if obj_type in CRYPTO_OBJECT_TYPES or (not obj_type and "-" in symbol and symbol.endswith("USD")):
                if not include_crypto:
                    skipped_crypto.append(symbol)
                    continue
            if obj_type in INDEX_OBJECT_TYPES:
indexes[symbol] = {"symbol": symbol, "object_type": "index", "sources": sources[symbol]}
            elif obj_type in OPTION_OBJECT_TYPES:
                # Options watchlist handled separately; still capture underlying if present.
                und = _normalize_symbol(item.get("chain_symbol") or item.get("underlying_symbol"))
                if und:
equities.setdefault(und, {"symbol": und, "object_type": "instrument", "sources": sources.get(und,
[str(name)])})
            else:
                # Default: treat as equity/ETF instrument.
equities[symbol] = {"symbol": symbol, "object_type": "instrument", "sources": sources[symbol]}

    option_underlyings: list[str] = []
    if include_options_watchlist and option_watchlist_items:
        for item in option_watchlist_items:
            und = _normalize_symbol(
                item.get("chain_symbol")
                or item.get("underlying_symbol")
                or item.get("symbol")
            )
            if und:
                option_underlyings.append(und)
```

```
                equities.setdefault(
                    und,
                    {"symbol": und, "object_type": "instrument", "sources": ["Options Watchlist"]},
                )

    return {
        "equities": sorted(equities.values(), key=lambda x: x["symbol"]),
        "indexes": sorted(indexes.values(), key=lambda x: x["symbol"]),
        "option_watchlist_underlyings": sorted(set(option_underlyings)),
        "skipped_crypto": sorted(set(skipped_crypto)),
        "equity_symbols": sorted(equities.keys()),
        "index_symbols": sorted(indexes.keys()),
    }


def apply_liquidity_filter(
    symbols: list[str],
    fundamentals_by_symbol: dict[str, dict[str, Any]],
    *,
    min_average_volume: float,
) -> dict[str, Any]:
    """Filter equities by average volume from fundamentals payloads."""
    passed: list[dict[str, Any]] = []
    rejected: list[dict[str, Any]] = []

    for symbol in symbols:
        fund = fundamentals_by_symbol.get(symbol) or {}
        # RH fundamentals field names can vary; accept common keys only if present.
        avg_vol = None
        for key in (
            "average_volume",
            "average_volume_2_weeks",
            "avg_volume",
            "volume",
        ):
            if key in fund and fund[key] not in (None, ""):
                try:
                    avg_vol = float(fund[key])
                    break
                except (TypeError, ValueError):
                    continue
        if avg_vol is None:
rejected.append({"symbol": symbol, "reason": "missing_average_volume", "average_volume": None})
            continue
        if avg_vol < min_average_volume:
            rejected.append(
                {
                    "symbol": symbol,
                    "reason": "below_min_average_volume",
                    "average_volume": avg_vol,
                    "min_average_volume": min_average_volume,
                }
            )
            continue
        passed.append({"symbol": symbol, "average_volume": avg_vol})

    return {
        "passed": passed,
        "rejected": rejected,
        "passed_symbols": [p["symbol"] for p in passed],
    }


def option_quote_liquid(
    quote: dict[str, Any],
    *,
    max_spread_pct_of_price: float,
    preferred_spread_pct_of_price: float = 0.05,
    reject_one_sided: bool = True,
) -> tuple[bool, str | None, dict[str, Any]]:
    """Return (ok, reason, metrics) for an option quote liquidity gate.

    Spread is measured as (ask · bid) / mid. Prefer · 5% of price; reject above 10%.
    There is no absolute-dollar override.
```

```
    """
    try:
        bid = float(quote.get("bid_price") or 0)
        ask = float(quote.get("ask_price") or 0)
    except (TypeError, ValueError):
        return False, "invalid_bid_ask", {}

    if reject_one_sided and (bid <= 0 or ask <= 0):
        return False, "one_sided_or_missing_quote", {"bid": bid, "ask": ask}

    mid = (bid + ask) / 2.0
    spread = ask - bid
    spread_pct = (spread / mid) if mid > 0 else None
    metrics: dict[str, Any] = {
        "bid": bid,
        "ask": ask,
        "mid": mid,
        "spread": spread,
        "spread_pct_of_price": spread_pct,
        "spread_pct_of_mid": spread_pct,
        "preferred_spread_pct_of_price": preferred_spread_pct_of_price,
        "max_spread_pct_of_price": max_spread_pct_of_price,
    }

    if mid <= 0:
        return False, "non_positive_mid", metrics
    if spread_pct is None:
        return False, "spread_too_wide", metrics
    if spread_pct <= preferred_spread_pct_of_price:
        metrics["spread_quality"] = "preferred"
        return True, None, metrics
    if spread_pct <= max_spread_pct_of_price:
        metrics["spread_quality"] = "acceptable"
        return True, None, metrics
    metrics["spread_quality"] = "too_wide"
    return False, "spread_too_wide", metrics
```

```python
from __future__ import annotations

from typing import Any

import numpy as np


def _local_extrema(prices: np.ndarray, order: int = 3) -> tuple[list[int], list[int]]:
    """Simple peak/trough detection with `order` bars on each side."""
    peaks: list[int] = []
    troughs: list[int] = []
    n = len(prices)
    for i in range(order, n - order):
        window = prices[i - order : i + order + 1]
        if prices[i] == window.max() and np.sum(window == prices[i]) == 1:
            peaks.append(i)
        if prices[i] == window.min() and np.sum(window == prices[i]) == 1:
            troughs.append(i)
    return peaks, troughs


def _nearly_equal(a: float, b: float, tol_pct: float = 0.015) -> bool:
    base = max(abs(a), abs(b), 1e-9)
    return abs(a - b) / base <= tol_pct


def detect_patterns(ohlc: list[dict[str, Any]], *, timeframe: str) -> list[dict[str, Any]]:
    """
    Deterministic pattern heuristics on close prices.
    Returns pattern hits with indices; empty if insufficient bars.
    """
    if len(ohlc) < 30:
        return []

    closes = np.array([float(b["close"]) for b in ohlc], dtype=float)
    highs = np.array([float(b.get("high", b["close"])) for b in ohlc], dtype=float)
    lows = np.array([float(b.get("low", b["close"])) for b in ohlc], dtype=float)
    peaks, troughs = _local_extrema(closes, order=3)
    hits: list[dict[str, Any]] = []

    # Double / triple tops
    if len(peaks) >= 2:
        p1, p2 = peaks[-2], peaks[-1]
        if _nearly_equal(closes[p1], closes[p2]):
            neck = float(closes[p1:p2].min()) if p2 > p1 else float(closes[p2])
            hits.append(
                {
                    "pattern": "double_top",
                    "timeframe": timeframe,
                    "indices": [p1, p2],
                    "prices": [float(closes[p1]), float(closes[p2])],
                    "neckline": neck,
                    "bias": "bearish",
                }
            )
    if len(peaks) >= 3:
        p1, p2, p3 = peaks[-3], peaks[-2], peaks[-1]
        if _nearly_equal(closes[p1], closes[p2]) and _nearly_equal(closes[p2], closes[p3]):
            hits.append(
                {
                    "pattern": "triple_top",
                    "timeframe": timeframe,
                    "indices": [p1, p2, p3],
                    "prices": [float(closes[p1]), float(closes[p2]), float(closes[p3])],
                    "bias": "bearish",
                }
            )

    # Double / triple bottoms
    if len(troughs) >= 2:
        t1, t2 = troughs[-2], troughs[-1]
        if _nearly_equal(closes[t1], closes[t2]):
            neck = float(closes[t1:t2].max()) if t2 > t1 else float(closes[t2])
            hits.append(
```

```
                {
                    "pattern": "double_bottom",
                    "timeframe": timeframe,
                    "indices": [t1, t2],
                    "prices": [float(closes[t1]), float(closes[t2])],
                    "neckline": neck,
                    "bias": "bullish",
                }
            )
    if len(troughs) >= 3:
        t1, t2, t3 = troughs[-3], troughs[-2], troughs[-1]
        if _nearly_equal(closes[t1], closes[t2]) and _nearly_equal(closes[t2], closes[t3]):
            hits.append(
                {
                    "pattern": "triple_bottom",
                    "timeframe": timeframe,
                    "indices": [t1, t2, t3],
                    "prices": [float(closes[t1]), float(closes[t2]), float(closes[t3])],
                    "bias": "bullish",
                }
            )

    # Head and shoulders / inverse (last three extrema of opposite type)
    if len(peaks) >= 3:
        l, h, r = peaks[-3], peaks[-2], peaks[-1]
if closes[h] > closes[l] and closes[h] > closes[r] and _nearly_equal(closes[l], closes[r],
tol_pct=0.025):
            hits.append(
                {
                    "pattern": "head_and_shoulders",
                    "timeframe": timeframe,
                    "indices": [l, h, r],
                    "prices": [float(closes[l]), float(closes[h]), float(closes[r])],
                    "bias": "bearish",
                }
            )
    if len(troughs) >= 3:
        l, h, r = troughs[-3], troughs[-2], troughs[-1]
if closes[h] < closes[l] and closes[h] < closes[r] and _nearly_equal(closes[l], closes[r],
tol_pct=0.025):
            hits.append(
                {
                    "pattern": "inverse_head_and_shoulders",
                    "timeframe": timeframe,
                    "indices": [l, h, r],
                    "prices": [float(closes[l]), float(closes[h]), float(closes[r])],
                    "bias": "bullish",
                }
            )

    # Triangles on recent 40 bars: converging highs/lows
    window = min(40, len(closes))
    seg_high = highs[-window:]
    seg_low = lows[-window:]
    x = np.arange(window, dtype=float)
    if window >= 20:
        high_slope = float(np.polyfit(x, seg_high, 1)[0])
        low_slope = float(np.polyfit(x, seg_low, 1)[0])
        high_range = float(seg_high.max() - seg_high.min())
        low_range = float(seg_low.max() - seg_low.min())
        flat_high = abs(high_slope) < (high_range / window) * 0.15
        flat_low = abs(low_slope) < (low_range / window) * 0.15
        rising_low = low_slope > 0
        falling_high = high_slope < 0
        if flat_high and rising_low:
            hits.append(
                {
                    "pattern": "ascending_triangle",
                    "timeframe": timeframe,
                    "high_slope": high_slope,
                    "low_slope": low_slope,
                    "bias": "bullish",
                }
            )
```

```
147         elif flat_low and falling_high:
148             hits.append(
149                 {
150                     "pattern": "descending_triangle",
151                     "timeframe": timeframe,
152                     "high_slope": high_slope,
153                     "low_slope": low_slope,
154                     "bias": "bearish",
155                 }
156             )
157         elif falling_high and rising_low:
158             hits.append(
159                 {
160                     "pattern": "symmetrical_triangle",
161                     "timeframe": timeframe,
162                     "high_slope": high_slope,
163                     "low_slope": low_slope,
164                     "bias": "neutral",
165                 }
166             )
167 
168     return hits
```

```python
"""Normalize Robinhood OHLCV bars and build custom intervals.

Robinhood MCP `get_equity_historicals` fixed intervals:
  15second, 30second, minute, 5minute, 10minute, 30minute, hour, 4hour, day, ...
The 1-minute bar is named `minute` (not `1minute`). There is no `3minute`
and no `15minute`. For 3-minute graphs, fetch `minute` and aggregate here.
"""

from __future__ import annotations

from datetime import datetime, timezone
from typing import Any


def _as_float(value: Any, default: float | None = None) -> float:
    if value is None or value == "":
        if default is None:
            raise ValueError("missing numeric bar field")
        return default
    return float(value)


def _begins_at(bar: dict[str, Any]) -> str:
    raw = bar.get("begins_at") or bar.get("timestamp") or bar.get("start") or bar.get("time") or ""
    return str(raw)


def _parse_utc(ts: str) -> datetime:
    text = ts.strip()
    if text.endswith("Z"):
        text = text[:-1] + "+00:00"
    dt = datetime.fromisoformat(text)
    if dt.tzinfo is None:
        dt = dt.replace(tzinfo=timezone.utc)
    return dt.astimezone(timezone.utc)


def normalize_bar(bar: dict[str, Any]) -> dict[str, Any]:
    """Accept RH (`open_price`) or pipeline (`open`) keys. Skip nothing here."""
    open_px = bar.get("open", bar.get("open_price"))
    high_px = bar.get("high", bar.get("high_price"))
    low_px = bar.get("low", bar.get("low_price"))
    close_px = bar.get("close", bar.get("close_price"))
    return {
        "begins_at": _begins_at(bar),
        "open": _as_float(open_px),
        "high": _as_float(high_px),
        "low": _as_float(low_px),
        "close": _as_float(close_px),
        "volume": _as_float(bar.get("volume"), default=0.0),
        "interpolated": bool(bar.get("interpolated")),
        "session": bar.get("session"),
    }


def normalize_bars(bars: list[dict[str, Any]] | None, *, drop_interpolated: bool = True) ->
list[dict[str, Any]]:
    out: list[dict[str, Any]] = []
    for bar in bars or []:
        try:
            row = normalize_bar(bar)
        except (TypeError, ValueError):
            continue
        if drop_interpolated and row["interpolated"]:
            continue
        out.append(row)
    out.sort(key=lambda b: b["begins_at"])
    return out


def extract_rh_historicals_bars(payload: Any) -> list[dict[str, Any]]:
    """Unwrap `get_equity_historicals` MCP JSON to a flat bar list."""
    if isinstance(payload, list):
        if payload and isinstance(payload[0], dict) and (
```

```
            "open" in payload[0] or "open_price" in payload[0] or "close" in payload[0]
        ):
            return payload
    data = payload
    if isinstance(payload, dict) and "data" in payload:
        data = payload["data"]
    results = []
    if isinstance(data, dict):
        results = data.get("results") or data.get("historicals") or []
        if isinstance(data.get("bars"), list):
            return data["bars"]
    bars: list[dict[str, Any]] = []
    for row in results:
        if not isinstance(row, dict):
            continue
        chunk = row.get("bars") or row.get("historicals") or []
        bars.extend(chunk)
    return bars


def aggregate_to_minutes(bars: list[dict[str, Any]] | None, minutes: int) -> list[dict[str, Any]]:
    """Build N-minute OHLCV from 1-minute (or finer) left-edge bars.

    Buckets align to UTC clock minutes (RTH 09:30 ET = 13:30 UTC, which is
    divisible by 3). Partial last buckets are kept (live in-progress bar).
    """
    if minutes < 1:
        raise ValueError("minutes must be >= 1")
    norm = normalize_bars(bars)
    if minutes == 1:
        return norm
    buckets: dict[datetime, list[dict[str, Any]]] = {}
    order: list[datetime] = []
    for bar in norm:
        if not bar["begins_at"]:
            continue
        dt = _parse_utc(bar["begins_at"]).replace(second=0, microsecond=0)
        minute_of_day = dt.hour * 60 + dt.minute
        aligned = minute_of_day - (minute_of_day % minutes)
        key = dt.replace(hour=aligned // 60, minute=aligned % 60)
        if key not in buckets:
            buckets[key] = []
            order.append(key)
        buckets[key].append(bar)
    out: list[dict[str, Any]] = []
    for key in order:
        group = buckets[key]
        out.append(
            {
                "begins_at": key.strftime("%Y-%m-%dT%H:%M:%SZ"),
                "open": group[0]["open"],
                "high": max(b["high"] for b in group),
                "low": min(b["low"] for b in group),
                "close": group[-1]["close"],
                "volume": sum(b["volume"] for b in group),
                "interpolated": False,
                "session": group[0].get("session"),
            }
        )
    return out


def bars_for_timeframe(
    historicals_for_symbol: dict[str, Any] | None,
    timeframe: str,
) -> list[dict[str, Any]]:
    """Resolve bars for a rules.json timeframe, synthesizing 3-minute from 1-minute."""
    by_tf = historicals_for_symbol or {}
    if timeframe == "3minute":
        existing = normalize_bars(by_tf.get("3minute") or [])
        if existing:
            return existing
        return aggregate_to_minutes(by_tf.get("minute") or by_tf.get("1minute") or [], 3)
    return normalize_bars(by_tf.get(timeframe) or [])
```

```python
"""Compact ASCII candlestick graphs for live / 1m / 3m / 5m bars."""

from __future__ import annotations

from typing import Any

from pipeline.bars import normalize_bars


def ascii_chart(
    bars: list[dict[str, Any]] | None,
    *,
    title: str,
    last_n: int = 48,
    height: int = 10,
) -> str:
    rows = normalize_bars(bars)[-last_n:]
    if not rows:
        return f"{title}: no bars"
    hi = max(b["high"] for b in rows)
    lo = min(b["low"] for b in rows)
    span = hi - lo or 1.0
    grid = [[" " for _ in rows] for _ in range(height)]

    def y_of(price: float) -> int:
        return max(0, min(height - 1, int(round((price - lo) / span * (height - 1)))))

    for i, bar in enumerate(rows):
        y_high = y_of(bar["high"])
        y_low = y_of(bar["low"])
        y_open = y_of(bar["open"])
        y_close = y_of(bar["close"])
        for y in range(min(y_low, y_high), max(y_low, y_high) + 1):
            grid[height - 1 - y][i] = "·"
        body_lo, body_hi = min(y_open, y_close), max(y_open, y_close)
        fill = "·" if bar["close"] >= bar["open"] else "·"
        for y in range(body_lo, body_hi + 1):
            grid[height - 1 - y][i] = fill

    last = rows[-1]
    first_ts = rows[0]["begins_at"]
    last_ts = last["begins_at"]
    lines = [
        title,
        f"{hi:.2f}",
        *["".join(row) for row in grid],
        f"{lo:.2f}  n={len(rows)}  {first_ts} · {last_ts}",
        (
            f"last O={last['open']:.2f} H={last['high']:.2f} "
            f"L={last['low']:.2f} C={last['close']:.2f} V={last['volume']:.0f}"
        ),
    ]
    return "\n".join(lines)


def live_quote_line(quote: dict[str, Any] | None, *, symbol: str) -> str:
    q = quote or {}
    inner = q.get("quote") if isinstance(q.get("quote"), dict) else q
    last = inner.get("last_trade_price") or inner.get("last_non_reg_trade_price") or inner.get("last")
    bid = inner.get("bid_price") or inner.get("bid")
    ask = inner.get("ask_price") or inner.get("ask")
    ts = (
        inner.get("venue_last_trade_time")
        or inner.get("venue_last_non_reg_trade_time")
        or inner.get("updated_at")
        or ""
    )
    return f"{symbol} live last={last} bid={bid} ask={ask} ts={ts}"
```

```python
from __future__ import annotations

from typing import Any

from pipeline.io_util import utc_now_iso


def build_news_signal(
    symbol: str,
    articles: list[dict[str, Any]],
    earnings: dict[str, Any] | None = None,
) -> dict[str, Any]:
    """Agent C: factual packaging of RH news/earnings payloads only."""
    headlines = []
    for a in articles[:10]:
        headlines.append(
            {
                "title": a.get("title") or a.get("headline"),
                "published_at": a.get("published_at") or a.get("updated_at") or a.get("created_at"),
                "source": a.get("source") or a.get("author"),
                "url": a.get("url") or a.get("link"),
            }
        )
    return {
        "symbol": symbol,
        "as_of": utc_now_iso(),
        "headline_count": len(headlines),
        "headlines": headlines,
        "earnings": earnings,
        "notes": "Read-only catalyst pack; no sentiment scores invented.",
    }
```

```python
from __future__ import annotations

from datetime import date, datetime
from typing import Any


def _parse_ymd(value: str) -> date:
    return datetime.strptime(value[:10], "%Y-%m-%d").date()


def dte(expiration: str, as_of: date | None = None) -> int:
    as_of = as_of or date.today()
    return (_parse_ymd(expiration) - as_of).days


def pick_atm_or_one_otm(
    spot: float,
    instruments: list[dict[str, Any]],
    *,
    option_type: str,
) -> dict[str, Any] | None:
    """Prefer ATM strike; fallback one strike OTM for calls/puts."""
    typed = [i for i in instruments if (i.get("type") or "").lower() == option_type.lower()]
    if not typed or spot <= 0:
        return None
    typed = sorted(typed, key=lambda i: abs(float(i["strike_price"]) - spot))
    atm = typed[0]
    atm_strike = float(atm["strike_price"])
    if abs(atm_strike - spot) / spot <= 0.01:
        return {"selection": "atm", "instrument": atm}

    if option_type.lower() == "call":
        otm = [i for i in typed if float(i["strike_price"]) > spot]
        otm = sorted(otm, key=lambda i: float(i["strike_price"]))
    else:
        otm = [i for i in typed if float(i["strike_price"]) < spot]
        otm = sorted(otm, key=lambda i: float(i["strike_price"]), reverse=True)

    if otm:
        return {"selection": "one_otm", "instrument": otm[0]}
    return {"selection": "atm_fallback", "instrument": atm}


def choose_structure_from_bias(bias: str | None) -> str | None:
    if bias == "bullish":
        return "long_call"
    if bias == "bearish":
        return "long_put"
    return None


def filter_expirations(expiration_dates: list[str], *, max_dte: int, as_of: date | None = None) ->
list[str]:
    as_of = as_of or date.today()
    out = []
    for exp in expiration_dates:
        days = dte(exp, as_of=as_of)
        if 0 <= days <= max_dte:
            out.append(exp)
    return sorted(out)
```

```python
from __future__ import annotations

from math import floor
from typing import Any

from pipeline.risk import equity_risk_plan

# Inverse / leveraged-short ETFs. Long index ETFs (SPY, VTI, QQQ) are allowed.
INVERSE_ETF_SYMBOLS = frozenset(
    {
        "SH",
        "SDS",
        "SPXU",
        "SPXS",
        "SPDN",
        "PSQ",
        "QID",
        "SQQQ",
        "DOG",
        "DXD",
        "SDOW",
        "TZA",
        "FAZ",
        "SOXS",
        "LABD",
        "YANG",
        "SCO",
        "DUG",
        "DUST",
        "JDST",
        "WEBS",
        "HIBS",
        "SARK",
        "RWM",
        "TWM",
        "HDGE",
        "EFZ",
        "EPV",
        "MYY",
        "MZZ",
        "BIS",
        "SRTY",
        "TYO",
        "KOLD",
        "SBB",
        "SEF",
        "SIJ",
        "SKF",
    }
)


def is_inverse_etf(symbol: str, fundamentals: dict[str, Any] | None = None) -> bool:
    if (symbol or "").strip().upper() in INVERSE_ETF_SYMBOLS:
        return True
    fund = fundamentals or {}
    blob = " ".join(
        str(fund.get(k) or "")
        for k in ("description", "name", "security_name", "instrument_name")
    ).lower()
    return "inverse" in blob


def parse_bid_ask(quote: dict[str, Any] | None) -> tuple[float | None, float | None]:
    if not quote:
        return None, None

    def _num(*keys: str) -> float | None:
        for key in keys:
            if key in quote and quote[key] not in (None, ""):
                try:
                    return float(quote[key])
                except (TypeError, ValueError):
                    continue
```

```
        return None

    bid = _num("bid_price", "bid", "bid_last")
    ask = _num("ask_price", "ask", "ask_last")
    return bid, ask


def equity_quote_ok(
    quote: dict[str, Any] | None,
    *,
    reject_one_sided: bool = True,
) -> tuple[bool, str | None, dict[str, Any]]:
    bid, ask = parse_bid_ask(quote)
    metrics: dict[str, Any] = {"bid": bid, "ask": ask}
    if bid is None or ask is None:
        return False, "missing_bid_ask", metrics
    if reject_one_sided and (bid <= 0 or ask <= 0):
        return False, "one_sided_or_missing_quote", metrics
    if ask < bid:
        return False, "crossed_quote", metrics
    mid = (bid + ask) / 2.0
    metrics["mid"] = mid
    metrics["spread"] = ask - bid
    return True, None, metrics


def regular_hours_buy_ok(tradability: dict[str, Any] | None) -> tuple[bool, str | None]:
    if not tradability:
        return False, "missing_tradability"
    if tradability.get("halted") is True:
        return False, "halted"
    rh = (
        tradability.get("regular_hours")
        or tradability.get("session_regular_hours")
        or tradability.get("regular")
        or {}
    )
    if isinstance(rh, dict) and rh:
        buy = rh.get("buy")
        if buy is False:
            return False, "regular_hours_buy_false"
        if buy is True:
            return True, None
        tradable = rh.get("tradable")
        if tradable is False:
            return False, "regular_hours_not_tradable"
        if tradable is True:
            return True, None
    for key in ("tradable", "is_tradable", "can_trade"):
        if tradability.get(key) is False:
            return False, "not_tradable"
        if tradability.get(key) is True:
            return True, None
    return False, "regular_hours_buy_not_confirmed"


def whole_share_size(buying_power: float, limit_price: float) -> dict[str, Any]:
    """shares = floor(buying_power / limit). Notional must be · buying power."""
    if buying_power is None or limit_price is None:
        return {"ok": False, "shares": 0, "notional": 0.0, "reason": "invalid_price_or_buying_power"}
    try:
        bp = float(buying_power)
        limit = float(limit_price)
    except (TypeError, ValueError):
        return {"ok": False, "shares": 0, "notional": 0.0, "reason": "invalid_price_or_buying_power"}
    if bp <= 0 or limit <= 0:
        return {"ok": False, "shares": 0, "notional": 0.0, "reason": "invalid_price_or_buying_power"}
    shares = int(floor(bp / limit))
    notional = shares * limit
    if shares < 1:
        return {"ok": False, "shares": 0, "notional": 0.0, "reason": "cannot_afford_one_share"}
    if notional > bp + 1e-9:
return {"ok": False, "shares": shares, "notional": notional, "reason": "notional_exceeds_buying_power"}
    return {"ok": True, "shares": shares, "notional": notional, "reason": None}
```

```
149
150
151 def buying_power_from_raw(raw: dict[str, Any]) -> float | None:
152     portfolio = raw.get("portfolio") if isinstance(raw.get("portfolio"), dict) else {}
153     for value in (
154         raw.get("buying_power"),
155         portfolio.get("buying_power"),
156         portfolio.get("buying_power_usd"),
157     ):
158         if value not in (None, ""):
159             try:
160                 return float(value)
161             except (TypeError, ValueError):
162                 continue
163     return None
164
165
166 def select_equity_day_trade_candidates(
167     *,
168     symbols: list[str],
169     technicals_by_symbol: dict[str, Any],
170     option_candidate_symbols: set[str],
171     quotes_by_symbol: dict[str, Any],
172     tradability_by_symbol: dict[str, Any],
173     fundamentals_by_symbol: dict[str, Any],
174     buying_power: float | None,
175     playbook_status: str,
176     take_profit_pct: float,
177     stop_loss_pct: float,
178 ) -> tuple[list[dict[str, Any]], list[dict[str, Any]]]:
179     """Bullish long-share day-trade candidates. Never shorts. Options-first skip."""
180     candidates: list[dict[str, Any]] = []
181     rejected: list[dict[str, Any]] = []
182
183     for symbol in symbols:
184         bias = (technicals_by_symbol.get(symbol) or {}).get("dominant_bias")
185         fund = fundamentals_by_symbol.get(symbol) or {}
186         if is_inverse_etf(symbol, fund):
187             rejected.append({"symbol": symbol, "reason": "inverse_etf", "bias": bias})
188             continue
189         if symbol in option_candidate_symbols:
190             rejected.append({"symbol": symbol, "reason": "options_priority", "bias": bias})
191             continue
192         if bias != "bullish":
193 rejected.append({"symbol": symbol, "reason": "equity_long_only_requires_bullish", "bias": bias})
194             continue
195         ok_tr, tr_reason = regular_hours_buy_ok(tradability_by_symbol.get(symbol))
196         if not ok_tr:
197             rejected.append({"symbol": symbol, "reason": tr_reason, "bias": bias})
198             continue
199         ok_q, q_reason, q_metrics = equity_quote_ok(quotes_by_symbol.get(symbol))
200         if not ok_q:
201             rejected.append({"symbol": symbol, "reason": q_reason, "bias": bias, **q_metrics})
202             continue
203         if buying_power is None:
204             rejected.append({"symbol": symbol, "reason": "missing_buying_power", "bias": bias})
205             continue
206         limit = float(q_metrics["ask"])
207         size = whole_share_size(buying_power, limit)
208         if not size["ok"]:
209 rejected.append({"symbol": symbol, "reason": size["reason"], "bias": bias, "limit": limit,
    "buying_power": buying_power})
210             continue
211         plan = equity_risk_plan(
212             cost_basis=float(size["notional"]),
213             take_profit_pct=take_profit_pct,
214             stop_loss_pct=stop_loss_pct,
215             shares=size["shares"],
216             limit_price=limit,
217         )
218         candidates.append(
219             {
220                 "symbol": symbol,
221                 "structure": "long_shares",
```

```
                "side": "buy",
                "bias": bias,
                "quantity": size["shares"],
                "limit_price": limit,
                "notional": size["notional"],
                "buying_power": buying_power,
                "bid": q_metrics["bid"],
                "ask": q_metrics["ask"],
                "order_type": "limit",
                "time_in_force": "gfd",
                "market_hours": "regular_hours",
                "playbook_status": playbook_status,
                "risk": plan,
            }
        )
    return candidates, rejected
```

```python
from __future__ import annotations

from typing import Any


def extract_greeks(quote: dict[str, Any]) -> dict[str, Any]:
    """Copy Greeks only from Robinhood quote fields; never invent values."""
    fields = (
        "delta",
        "gamma",
        "theta",
        "vega",
        "rho",
        "implied_volatility",
    )
    out: dict[str, Any] = {}
    missing: list[str] = []
    for f in fields:
        if f in quote and quote[f] not in (None, ""):
            try:
                out[f] = float(quote[f])
            except (TypeError, ValueError):
                missing.append(f)
        else:
            missing.append(f)
    return {"greeks": out, "missing_fields": missing, "source": "robinhood_get_option_quotes"}


def delta_in_band(delta: float | None, *, lo: float, hi: float) -> tuple[bool, str | None]:
    if delta is None:
        return False, "delta_missing_from_quote"
    # Calls: positive delta. Puts: negative · compare absolute value for long options band.
    ad = abs(delta)
    if lo <= ad <= hi:
        return True, None
    return False, f"delta_abs_{ad:.4f}_outside_{lo}_{hi}"
```

```python
from __future__ import annotations

from typing import Any

OPTIONS_SL_PCT_MIN = 0.20
OPTIONS_SL_PCT_MAX = 0.50
OPTIONS_TP_PCT_MIN = 0.30
OPTIONS_TARGET_REWARD_TO_RISK = 2.0
OPTIONS_DEFAULT_SL_PCT = 0.20
OPTIONS_DEFAULT_TP_PCT = 0.40


def options_risk_plan(
    *,
    premium_per_share: float,
    contracts: int = 1,
    multiplier: float = 100.0,
    take_profit_pct: float = OPTIONS_DEFAULT_TP_PCT,
    stop_loss_pct: float = OPTIONS_DEFAULT_SL_PCT,
) -> dict[str, Any]:
    """Cash-risked plan for long options. Does not invent prices beyond inputs.

    Locked bands: SL 20·50% of premium; TP 30·100%+ of premium; aim 1:2 R:R.
    Owner-locked working pair is ·20% / +40% (1:2, inside the bands).
    """
    if contracts != 1:
        raise ValueError("Phase rules require max 1 contract")
    if stop_loss_pct < OPTIONS_SL_PCT_MIN or stop_loss_pct > OPTIONS_SL_PCT_MAX:
        raise ValueError(
f"options stop_loss_pct must be in [{OPTIONS_SL_PCT_MIN}, {OPTIONS_SL_PCT_MAX}], got {stop_loss_pct}"
        )
    if take_profit_pct < OPTIONS_TP_PCT_MIN:
        raise ValueError(
            f"options take_profit_pct must be >= {OPTIONS_TP_PCT_MIN}, got {take_profit_pct}"
        )
    cash = premium_per_share * multiplier * contracts
    reward_to_risk = take_profit_pct / stop_loss_pct if stop_loss_pct else None
    return {
        "asset_class": "option",
        "contracts": contracts,
        "premium_per_share": premium_per_share,
        "multiplier": multiplier,
        "cash_risked": cash,
        "take_profit_pct": take_profit_pct,
        "stop_loss_pct": stop_loss_pct,
        "take_profit_value": cash * (1.0 + take_profit_pct),
        "stop_loss_value": cash * (1.0 - stop_loss_pct),
        "take_profit_premium": premium_per_share * (1.0 + take_profit_pct),
        "stop_loss_premium": premium_per_share * (1.0 - stop_loss_pct),
        "stop_loss_pct_band": {"min": OPTIONS_SL_PCT_MIN, "max": OPTIONS_SL_PCT_MAX},
        "take_profit_pct_band": {"min": OPTIONS_TP_PCT_MIN, "uncapped": True},
        "target_reward_to_risk": OPTIONS_TARGET_REWARD_TO_RISK,
        "reward_to_risk": reward_to_risk,
        "meets_target_rr": reward_to_risk is not None
        and reward_to_risk + 1e-12 >= OPTIONS_TARGET_REWARD_TO_RISK,
        "broker_exit": "stop_first_until_oco",
        "monitor_take_profit_in_loop": True,
    }


def equity_risk_plan(
    *,
    cost_basis: float,
    take_profit_pct: float = 0.25,
    stop_loss_pct: float = 0.2,
    shares: int | None = None,
    limit_price: float | None = None,
) -> dict[str, Any]:
    plan: dict[str, Any] = {
        "asset_class": "equity",
        "cost_basis": cost_basis,
        "take_profit_pct": take_profit_pct,
        "stop_loss_pct": stop_loss_pct,
        "take_profit_value": cost_basis * (1.0 + take_profit_pct),
```

```
        "stop_loss_value": cost_basis * (1.0 - stop_loss_pct),
        "broker_exit": "stop_first_until_oco",
        "monitor_take_profit_in_loop": True,
        "flatten_before_close": True,
        "side": "long_only",
    }
    if shares is not None:
        plan["shares"] = int(shares)
    if limit_price is not None:
        plan["limit_price"] = limit_price
        plan["stop_price"] = limit_price * (1.0 - stop_loss_pct)
        plan["take_profit_price"] = limit_price * (1.0 + take_profit_pct)
    return plan
```

```python
from __future__ import annotations

from collections import Counter
from typing import Any

from pipeline.bars import bars_for_timeframe
from pipeline.equity_day_trade import buying_power_from_raw, select_equity_day_trade_candidates
from pipeline.greeks import delta_in_band, extract_greeks
from pipeline.io_util import append_jsonl, load_rules, utc_now_iso, write_json, SIGNALS, JOURNAL
from pipeline.news import build_news_signal
from pipeline.options_structure import (
    choose_structure_from_bias,
    filter_expirations,
    pick_atm_or_one_otm,
)
from pipeline.patterns import detect_patterns
from pipeline.risk import equity_risk_plan, options_risk_plan
from pipeline.universe import apply_liquidity_filter, extract_watchlist_symbols, option_quote_liquid


def dominant_bias(pattern_hits: list[dict[str, Any]]) -> str | None:
    biases = [p.get("bias") for p in pattern_hits if p.get("bias") in ("bullish", "bearish")]
    if not biases:
        return None
    counts = Counter(biases)
    # Require strict majority
    top, n = counts.most_common(1)[0]
    if n > len(biases) / 2:
        return top
    return None


def run_pipeline(raw: dict[str, Any]) -> dict[str, Any]:
    """
    Phase 2 orchestrator.
    `raw` is assembled by the Cursor agent from RH MCP responses (read-only).
    Never places orders.
    """
    rules = load_rules()
    as_of = utc_now_iso()
    assert rules["execution"]["phase2_places_orders"] is False

    # --- Agent A ---
    universe_extract = extract_watchlist_symbols(
        raw.get("watchlists", []),
        raw.get("watchlist_items_by_id", {}),
        raw.get("option_watchlist_items"),
        include_crypto=rules["universe"]["include_crypto"],
        include_options_watchlist=rules["universe"]["include_options_watchlist"],
    )
    liq = apply_liquidity_filter(
        universe_extract["equity_symbols"],
        raw.get("fundamentals_by_symbol", {}),
        min_average_volume=float(rules["liquidity"]["min_average_volume"]),
    )
    universe_signal = {
        "agent": "A_scanner",
        "as_of": as_of,
        "mode": "read_only",
        "extract": universe_extract,
        "liquidity": liq,
        "index_symbols": universe_extract["index_symbols"],
        "eligible_equities": liq["passed_symbols"],
    }
    write_json(SIGNALS / "universe.json", universe_signal)

    # --- Agent B ---
    technicals: dict[str, Any] = {"agent": "B_patterns", "as_of": as_of, "symbols": {}}
    historicals = raw.get("historicals_by_symbol_timeframe", {})
    for symbol in liq["passed_symbols"]:
        symbol_hits: list[dict[str, Any]] = []
        for tf in rules["patterns"]["timeframes"]:
            bars = bars_for_timeframe(historicals.get(symbol) or {}, tf)
            symbol_hits.extend(detect_patterns(bars, timeframe=tf))
```

```
        technicals["symbols"][symbol] = {
            "pattern_hits": symbol_hits,
            "dominant_bias": dominant_bias(symbol_hits),
        }
    write_json(SIGNALS / "technicals.json", technicals)

    # --- Agent C ---
    news_signal = {"agent": "C_news", "as_of": as_of, "symbols": {}}
    news_raw = raw.get("news_by_symbol", {})
    earnings_raw = raw.get("earnings_by_symbol", {})
    for symbol in liq["passed_symbols"]:
        news_signal["symbols"][symbol] = build_news_signal(
            symbol,
            news_raw.get(symbol) or [],
            earnings_raw.get(symbol),
        )
    write_json(SIGNALS / "news.json", news_signal)

    # --- Agents D + I ---
    option_candidates: list[dict[str, Any]] = []
    greeks_rows: list[dict[str, Any]] = []
    equity_fallbacks: list[dict[str, Any]] = []
    spots = raw.get("spots_by_symbol", {})
    chains = raw.get("option_chains_by_symbol", {})
    instruments_by_key = raw.get("option_instruments_by_symbol_exp", {})
    quotes_by_id = raw.get("option_quotes_by_id", {})

    for symbol in liq["passed_symbols"]:
        bias = technicals["symbols"].get(symbol, {}).get("dominant_bias")
        structure = choose_structure_from_bias(bias)
        spot = spots.get(symbol)
        if structure is None or spot is None:
            equity_fallbacks.append(
                {
                    "symbol": symbol,
                    "reason": "no_directional_bias_or_spot" if structure is None else "missing_spot",
                    "bias": bias,
                }
            )
            continue

        chain = chains.get(symbol) or {}
expirations = filter_expirations(chain.get("expiration_dates") or [],
max_dte=int(rules["options"]["max_dte"]))
        if not expirations:
equity_fallbacks.append({"symbol": symbol, "reason": "no_expiration_within_max_dte", "bias": bias})
            continue

        exp = expirations[0]
        option_type = "call" if structure == "long_call" else "put"
        instruments = instruments_by_key.get(f"{symbol}|{exp}") or instruments_by_key.get(symbol) or []
        pick = pick_atm_or_one_otm(float(spot), instruments, option_type=option_type)
        if not pick:
            equity_fallbacks.append({"symbol": symbol, "reason": "no_instrument_match", "bias": bias})
            continue

        inst = pick["instrument"]
        oid = inst["id"]
        quote = quotes_by_id.get(oid) or {}
        ok_liq, liq_reason, liq_metrics = option_quote_liquid(
            quote,
            max_spread_pct_of_price=float(rules["liquidity"]["option_max_spread_pct_of_price"]),
preferred_spread_pct_of_price=float(rules["liquidity"]["option_preferred_spread_pct_of_price"]),
            reject_one_sided=bool(rules["liquidity"]["reject_one_sided_quotes"]),
        )
        gpack = extract_greeks(quote)
        greeks_rows.append(
            {
                "symbol": symbol,
                "option_id": oid,
                "expiration": exp,
                "type": option_type,
                "strike": inst.get("strike_price"),
                **gpack,
```

```
                "liquidity": {"ok": ok_liq, "reason": liq_reason, **liq_metrics},
            }
        )
        delta = gpack["greeks"].get("delta")
        ok_delta, delta_reason = delta_in_band(
            delta,
            lo=float(rules["options"]["strike"]["delta_min"]),
            hi=float(rules["options"]["strike"]["delta_max"]),
        )
        if not ok_liq:
            equity_fallbacks.append(
{"symbol": symbol, "reason": f"option_illiquid:{liq_reason}", "bias": bias, "option_id": oid}
            )
            continue
        if not ok_delta:
            equity_fallbacks.append(
{"symbol": symbol, "reason": f"greeks_filter:{delta_reason}", "bias": bias, "option_id": oid}
            )
            continue

        mark = quote.get("mark_price") or quote.get("adjusted_mark_price")
        try:
            premium = float(mark)
        except (TypeError, ValueError):
equity_fallbacks.append({"symbol": symbol, "reason": "missing_mark_price", "option_id": oid})
            continue

        option_candidates.append(
            {
                "symbol": symbol,
                "structure": structure,
                "bias": bias,
                "expiration": exp,
                "option_type": option_type,
                "strike": inst.get("strike_price"),
                "option_id": oid,
                "selection": pick["selection"],
                "premium_mark": premium,
                "greeks": gpack["greeks"],
                "liquidity": liq_metrics,
                "contracts": 1,
                "playbook_status": rules["options"]["playbook_status"],
            }
        )

    write_json(
        SIGNALS / "option_candidates.json",
        {
            "agent": "D_option_structure",
            "as_of": as_of,
            "mode": "read_only",
            "candidates": option_candidates,
            "equity_fallbacks": equity_fallbacks,
            "max_contracts": rules["options"]["max_contracts"],
        },
    )
    write_json(
        SIGNALS / "greeks.json",
{"agent": "I_greeks", "as_of": as_of, "source": "robinhood_get_option_quotes", "rows": greeks_rows},
    )

    equity_candidates, equity_rejects = select_equity_day_trade_candidates(
        symbols=liq["passed_symbols"],
        technicals_by_symbol=technicals["symbols"],
        option_candidate_symbols={c["symbol"] for c in option_candidates},
        quotes_by_symbol=raw.get("equity_quotes_by_symbol") or {},
        tradability_by_symbol=raw.get("equity_tradability_by_symbol") or {},
        fundamentals_by_symbol=raw.get("fundamentals_by_symbol") or {},
        buying_power=buying_power_from_raw(raw),
        playbook_status=str(rules["risk"]["equity"]["playbook_status"]),
        take_profit_pct=float(rules["risk"]["equity"]["take_profit_pct_of_cost"]),
        stop_loss_pct=float(rules["risk"]["equity"]["stop_loss_pct_of_cost"]),
    )
    write_json(
```

```
222         SIGNALS / "equity_candidates.json",
223         {
224             "agent": "D_equity_day_trade",
225             "as_of": as_of,
226             "mode": "read_only",
227             "playbook_status": rules["risk"]["equity"]["playbook_status"],
228             "playbook_path": rules["risk"]["equity"]["playbook_path"],
229             "side": "long_only",
230             "no_shorting": True,
231             "priority": "options_first",
232             "candidates": equity_candidates,
233             "rejected": equity_rejects,
234             "option_fallback_notes": equity_fallbacks,
235         },
236     )
237
238     # --- Agent E ---
239     risk_plans: list[dict[str, Any]] = []
240     for cand in option_candidates:
241         plan = options_risk_plan(
242             premium_per_share=float(cand["premium_mark"]),
243             contracts=1,
244             take_profit_pct=float(rules["risk"]["options"]["take_profit_pct_of_cash"]),
245             stop_loss_pct=float(rules["risk"]["options"]["stop_loss_pct_of_cash"]),
246         )
247         risk_plans.append({"symbol": cand["symbol"], "option_id": cand["option_id"], **plan})
248
249     # Equity fallback risk plans only when explicitly provided cost basis in raw (optional)
250     for fb in raw.get("equity_fallback_costs", []) or []:
251         plan = equity_risk_plan(
252             cost_basis=float(fb["cost_basis"]),
253             take_profit_pct=float(rules["risk"]["equity"]["take_profit_pct_of_cost"]),
254             stop_loss_pct=float(rules["risk"]["equity"]["stop_loss_pct_of_cost"]),
255         )
256         risk_plans.append({"symbol": fb["symbol"], **plan})
257
258     for cand in equity_candidates:
259         risk_plans.append({"symbol": cand["symbol"], **cand["risk"]})
260
261     write_json(
262         SIGNALS / "risk_plan.json",
263         {
264             "agent": "E_risk",
265             "as_of": as_of,
266             "mode": "read_only",
267             "max_open_positions": rules["risk"]["max_open_positions"],
268             "plans": risk_plans,
269             "notes": "Stop-first until OCO; TP monitored in loop. Phase 2 does not place orders.",
270         },
271     )
272
273     summary = {
274         "as_of": as_of,
275         "phase": 2,
276         "places_orders": False,
277         "eligible_equities": liq["passed_symbols"],
278         "option_candidate_count": len(option_candidates),
279         "equity_candidate_count": len(equity_candidates),
280         "equity_fallback_count": len(equity_fallbacks),
281         "risk_plan_count": len(risk_plans),
282         "open_questions": rules.get("open_questions", []),
283     }
284     write_json(SIGNALS / "phase2_summary.json", summary)
285     append_jsonl(
286         JOURNAL / "loop_runs.jsonl",
287         {"event": "phase2_cycle", "mode": "read_only", **summary},
288     )
289     append_jsonl(
290         JOURNAL / f"{as_of[:10]}.md.jsonl",
291 {"type": "markdown_seed", "text": f"# {as_of[:10]} Phase 2 cycle\n\n- options candidates:
    {len(option_candidates)}\n- eligible equities: {len(liq['passed_symbols'])}\n"},
292     )
293     # Human-readable daily markdown journal
294     md_path = JOURNAL / f"{as_of[:10]}.md"
```

```
295  prev = md_path.read_text(encoding="utf-8") if md_path.exists() else f"# Trading journal
     {as_of[:10]}\n\n"
296      prev += f"\n## Phase 2 read-only cycle ({as_of})\n"
297      prev += f"- Eligible equities: {', '.join(liq['passed_symbols']) or '(none)'}\n"
298      prev += f"- Option candidates: {len(option_candidates)}\n"
299      prev += f"- Equity day-trade candidates: {len(equity_candidates)}\n"
300      prev += f"- Equity fallbacks (option miss): {len(equity_fallbacks)}\n"
301      prev += "- Orders placed: **none** (Phase 2 read-only)\n"
302      md_path.write_text(prev, encoding="utf-8")
303
304      return summary
```

```python
from __future__ import annotations

from datetime import datetime
from typing import Any

from pipeline.io_util import JOURNAL, SIGNALS, append_jsonl, load_rules, read_json, utc_now_iso,
write_json
from pipeline.session import is_rth


def load_latest_option_candidates() -> list[dict[str, Any]]:
    path = SIGNALS / "option_candidates.json"
    if not path.exists():
        return []
    payload = read_json(path)
    if payload.get("do_not_place") or payload.get("historical"):
        return []
    return list(payload.get("candidates") or [])


def load_latest_equity_candidates() -> list[dict[str, Any]]:
    path = SIGNALS / "equity_candidates.json"
    if not path.exists():
        return []
    payload = read_json(path)
    if payload.get("do_not_place") or payload.get("historical"):
        return []
    return list(payload.get("candidates") or [])


def can_place_live(
    *,
    explicit_confirm: bool,
    playbook_released: bool,
    playbook_kind: str = "options",
    h_enabled: bool | None = None,
    now: datetime | None = None,
    h_rth_override: bool = False,
) -> tuple[bool, str | None]:
    """Agent F place-gate. Never invent a confirm. H owns RTH while enabled."""
    if not playbook_released:
        return False, f"{playbook_kind}_playbook_still_draft"
    if not explicit_confirm:
        return False, "missing_explicit_user_confirm"
    if h_enabled is None:
        h_enabled = load_rules().get("execution", {}).get("unsupervised_agent_h") == "enabled"
    if h_enabled and is_rth(now) and not h_rth_override:
        return False, "h_owns_rth_while_enabled"
    return True, None


def build_option_entry_proposal(
    candidate: dict[str, Any],
    *,
    limit_price: str,
    account_last4: str = "2907",
) -> dict[str, Any]:
    """Build a supervised review/place payload. Does not call the broker."""
    return {
        "agent": "F_supervised_execution",
        "mode": "dry_review_until_confirm",
        "as_of": utc_now_iso(),
        "account_last4": account_last4,
        "asset_class": "option",
        "action": "buy_to_open",
        "symbol": candidate.get("symbol"),
        "structure": candidate.get("structure"),
        "expiration": candidate.get("expiration"),
        "option_type": candidate.get("option_type"),
        "strike": candidate.get("strike"),
        "option_id": candidate.get("option_id"),
        "quantity": "1",
        "order_type": "limit",
        "price": limit_price,
```

```
        "time_in_force": "gfd",
        "market_hours": "regular_hours",
        "chain_symbol": candidate.get("symbol"),
        "underlying_type": "equity",
        "legs": [
            {
                "option_id": candidate.get("option_id"),
                "side": "buy",
                "position_effect": "open",
                "ratio_quantity": 1,
            }
        ],
        "playbook_status": candidate.get("playbook_status"),
        "places_order": False,
    }


def build_equity_entry_proposal(
    candidate: dict[str, Any],
    *,
    limit_price: str | None = None,
    quantity: int | None = None,
    account_last4: str = "2907",
) -> dict[str, Any]:
    """Long-only equity day-trade proposal. Never sell-to-open. Does not call the broker."""
    shares = int(quantity if quantity is not None else candidate.get("quantity") or 0)
    if shares < 1:
        raise ValueError("equity day trade requires at least 1 whole share")
    price = str(limit_price if limit_price is not None else candidate.get("limit_price"))
    fill = float(price)
    stop_pct = float((candidate.get("risk") or {}).get("stop_loss_pct") or 0.2)
    tp_pct = float((candidate.get("risk") or {}).get("take_profit_pct") or 0.25)
    return {
        "agent": "F_supervised_execution",
        "mode": "dry_review_until_confirm",
        "as_of": utc_now_iso(),
        "account_last4": account_last4,
        "asset_class": "equity",
        "action": "buy_to_open",
        "side": "buy",
        "symbol": candidate.get("symbol"),
        "structure": "long_shares",
        "quantity": str(shares),
        "order_type": "limit",
        "price": price,
        "time_in_force": "gfd",
        "market_hours": "regular_hours",
        "stop_loss_pct": stop_pct,
        "take_profit_pct": tp_pct,
        "stop_price_after_fill": fill * (1.0 - stop_pct),
        "take_profit_price": fill * (1.0 + tp_pct),
        "playbook_status": candidate.get("playbook_status"),
        "places_order": False,
    }


def record_review(proposal: dict[str, Any], review_response: dict[str, Any] | None, *, error: str |
None = None) -> dict[str, Any]:
    rules = load_rules()
    asset = proposal.get("asset_class") or "option"
    if asset == "equity":
        released = rules["risk"]["equity"]["playbook_status"] == "RELEASED"
        kind = "equity"
        status = rules["risk"]["equity"]["playbook_status"]
    else:
        released = rules["options"]["playbook_status"] == "RELEASED"
        kind = "options"
        status = rules["options"]["playbook_status"]
    allowed, reason = can_place_live(
        explicit_confirm=False,
        playbook_released=released,
        playbook_kind=kind,
        h_enabled=rules.get("execution", {}).get("unsupervised_agent_h") == "enabled",
    )
```

```
147      record = {
148          "event": "phase3_dry_review",
149          "as_of": utc_now_iso(),
150          "proposal": proposal,
151          "review": review_response,
152          "error": error,
153          "places_order": False,
154          "playbook_status": status,
155          "place_gate": {"allowed": allowed, "reason": reason},
156      }
157      write_json(SIGNALS / "execution_review.json", record)
158      append_jsonl(JOURNAL / "reviews.jsonl", record)
159      return record
```

```
#!/usr/bin/env python3
"""Run Phase 2 pipeline from a raw MCP dump JSON file (read-only)."""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from pipeline.io_util import DATA_RAW
from pipeline.orchestrator import run_pipeline


def main() -> int:
    parser = argparse.ArgumentParser(description="Phase 2 read-only signal pipeline")
    parser.add_argument(
        "--raw",
        type=Path,
        default=DATA_RAW / "latest_raw.json",
        help="Path to RH MCP assembled raw JSON",
    )
    args = parser.parse_args()
    if not args.raw.exists():
        print(f"Raw file not found: {args.raw}", file=sys.stderr)
        return 1
    raw = json.loads(args.raw.read_text(encoding="utf-8"))
    summary = run_pipeline(raw)
    print(json.dumps(summary, indent=2))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

```
#!/usr/bin/env python3
"""Build a printable HTML + concatenated Python source book of this repo.

Covers every .py module used by Agent F (this Cursor chat) and Agent H
(the autonomous Agentic bot). H's place-permission prompt is not Python
and is listed only as a pointer.
"""

from __future__ import annotations

import html
from datetime import datetime, timezone
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
DOCS = ROOT / "docs"

# (path, part, used_by, one-line role)
CATALOG: list[tuple[str, str, str, str]] = [
    ("pipeline/__init__.py", "0 · Package", "F + H", "Pipeline package marker"),
    ("pipeline/io_util.py", "1 · Shared", "F + H", "Paths, rules.json loader, journal helpers"),
("pipeline/session.py", "1 · Shared", "F + H", "RTH clock: 09:30·16:00 ET, no new entries after 15:45"),
("pipeline/orders.py", "1 · Shared", "F + H", "Working-order states for Robinhood MCP (no open=true)"),
("pipeline/universe.py", "2 · Agent A", "F + H", "Watchlist extract, crypto drop, ADV · 2,000,000, option quote liquidity"),
    ("pipeline/patterns.py", "3 · Agent B", "F + H", "H&S, double/triple top/bottom, triangles"),
("pipeline/bars.py", "3 · Agent B", "F + H", "Normalize RH OHLCV; synthesize 3-minute from 1-minute"),
    ("pipeline/charts.py", "3 · Agent B", "F + H", "ASCII 1m / 3m / 5m candlestick graphs"),
("pipeline/news.py", "4 · Agent C", "F + H", "Factual RH news/earnings pack; no invented sentiment"),
("pipeline/options_structure.py", "5 · Agent D", "F + H", "Long call/put from bias; ATM else one OTM; DTE 0·7"),
("pipeline/equity_day_trade.py", "5 · Agent D", "F + H", "Long shares only; inverse-ETF denylist; size to buying power"),
    ("pipeline/greeks.py", "6 · Agent I", "F + H", "Copy RH Greeks only; abs(delta) 0.40·0.50 band"),
("pipeline/risk.py", "7 · Agent E", "F + H", "Options ·20%/+40%; equity ·20%/+25%; stop first until OCO"),
("pipeline/orchestrator.py", "8 · Agent G", "F + H", "Phase 2 read-only cycle; writes signals/; never places"),
("pipeline/execution.py", "9 · Agent F", "F (chat)", "Supervised review/place gate; blocked in RTH while H is on"),
("scripts/run_phase2_cycle.py", "10 · CLI", "F + H", "Load data/raw/latest_raw.json and run the orchestrator"),
("scripts/build_python_source_book.py", "10 · CLI", "docs", "This generator · rebuilds the printable source book"),
("pipeline/tests/test_phase2.py", "11 · Tests", "CI / F", "Universe, liquidity, greeks, ATM, risk math"),
("pipeline/tests/test_orders.py", "11 · Tests", "CI / F", "Working states and locked rules.json graph intervals"),
    ("pipeline/tests/test_bars.py", "11 · Tests", "CI / F", "1m·3m aggregation and live 5m match"),
("pipeline/tests/test_equity_day_trade.py", "11 · Tests", "CI / F", "Long-only equity day-trade selection"),
("pipeline/tests/test_execution.py", "11 · Tests", "CI / F", "F place-gate and historical snapshot skip"),
]


def _read(rel: str) -> str:
    path = ROOT / rel
    return path.read_text(encoding="utf-8") if path.exists() else f"# MISSING: {rel}\n"


def _banner(rel: str, part: str, used_by: str, role: str) -> str:
    line = "=" * 72
    return (
        f"# {line}\n"
        f"# {rel}\n"
        f"# Part: {part}\n"
        f"# Used by: {used_by}\n"
        f"# {role}\n"
        f"# {line}\n\n"
    )


```

```
 62  def build_python_book() -> str:
 63      now = datetime.now(timezone.utc).replace(microsecond=0).isoformat()
 64      chunks: list[str] = [
 65          "# ruff: noqa\n",
 66          "# pylint: skip-file\n",
 67          '"""PRINTABLE SOURCE BOOK · do not import or execute this file.\n',
 68          "\n",
 69          "Agentic trading program: Agent F (supervised Cursor chat) and\n",
 70          "Agent H (autonomous Agentic bot) share this Python pipeline.\n",
 71          "Live place_* is Robinhood MCP, not a side effect of this code.\n",
 72          "H's standing prompt is playbooks/agent_h_autonomous.PROMPT.md (not Python).\n",
 73          "\n",
 74          f"Generated: {now}\n",
 75          "Print companion: docs/agentic-python-source-printable.html\n",
 76          '"""\n\n',
 77      ]
 78      for rel, part, used_by, role in CATALOG:
 79          chunks.append(_banner(rel, part, used_by, role))
 80          chunks.append(_read(rel).rstrip() + "\n\n")
 81      return "".join(chunks)
 82
 83
 84  def build_html(py_book: str) -> str:
 85      now = datetime.now(timezone.utc).replace(microsecond=0).strftime("%Y-%m-%d %H:%M UTC")
 86      toc_rows = []
 87      sections = []
 88      total_lines = 0
 89      for rel, part, used_by, role in CATALOG:
 90          src = _read(rel)
 91          n = src.count("\n") + (0 if src.endswith("\n") or not src else 1)
 92          total_lines += n
 93          anchor = rel.replace("/", "-").replace(".", "-")
 94          toc_rows.append(
 95              f"<tr><td>{html.escape(part)}</td><td><a href='#{anchor}'>"
 96              f"<code>{html.escape(rel)}</code></a></td>"
 97              f"<td>{html.escape(used_by)}</td><td>{n}</td>"
 98              f"<td>{html.escape(role)}</td></tr>"
 99          )
100          numbered = []
101          lines = src.splitlines()
102          width = max(3, len(str(len(lines))))
103          for i, line in enumerate(lines, 1):
104              numbered.append(
105                  f"<span class='ln'>{i:{width}d}</span> {html.escape(line)}"
106              )
107          sections.append(
108              f"<section class='file' id='{anchor}'>"
109              f"<h2>{html.escape(rel)}</h2>"
110              f"<p class='meta'><strong>{html.escape(part)}</strong> · {html.escape(used_by)}"
111              f" · {n} lines · {html.escape(role)}</p>"
112              f"<pre class='source'>{chr(10).join(numbered)}\n</pre>"
113              f"</section>"
114          )
115
116      return f"""<!DOCTYPE html>
117  <html lang="en">
118  <head>
119    <meta charset="utf-8" />
120    <meta name="viewport" content="width=device-width, initial-scale=1" />
121    <title>Agentic Python Source Book (Printable)</title>
122    <style>
123      :root {{ --ink:#111; --muted:#333; --line:#222; --bg:#fff; --box:#f4f4f4; }}
124      * {{ box-sizing: border-box; }}
125      body {{
126        margin: 0; color: var(--ink); background: var(--bg);
127        font-family: "IBM Plex Sans", "Segoe UI", Helvetica, Arial, sans-serif;
128        font-size: 10.4pt; line-height: 1.35;
129      }}
130      .toolbar {{
131        position: sticky; top: 0; z-index: 20; display: flex; gap: 12px;
132        align-items: center; background: #111; color: #fff;
133        padding: 10px 16px; font-size: 13px;
134      }}
135      .toolbar button {{
```

```
      background: #fff; color: #111; border: 0; padding: 8px 14px;
      font-weight: 700; cursor: pointer;
    }}
    .page {{ max-width: 8.5in; margin: 0 auto; padding: 0.5in 0.55in 0.65in; }}
    h1 {{ font-size: 20pt; margin: 0 0 0.08in; letter-spacing: -0.02em; }}
    h2 {{
      font-size: 12pt; margin: 0.28in 0 0.08in; padding-bottom: 0.04in;
      border-bottom: 1.5pt solid var(--line); page-break-after: avoid;
    }}
    h3 {{ font-size: 11pt; margin: 0.16in 0 0.06in; page-break-after: avoid; }}
    .kicker {{ font-size: 9.5pt; letter-spacing: 0.08em; text-transform: uppercase; }}
    .rule {{ width: 1.3in; height: 3px; background: #111; margin: 0.12in 0 0.18in; }}
    .meta, .note, .footer {{ font-size: 9.3pt; color: var(--muted); }}
    .meta strong {{ color: var(--ink); }}
    .badge-row {{ display: flex; flex-wrap: wrap; gap: 6px; margin: 0.1in 0 0.16in; }}
    .badge {{
      border: 1pt solid var(--line); padding: 3px 8px; font-size: 8.4pt; background: var(--box);
    }}
    table {{ width: 100%; border-collapse: collapse; font-size: 8.8pt; margin: 0 0 0.14in; }}
th, td {{ border: 1pt solid var(--line); padding: 0.045in 0.06in; vertical-align: top; text-align:
left; }}
    th {{ background: #e8e8e8; font-weight: 700; }}
    .card {{
      border: 1pt solid var(--line); padding: 0.1in 0.12in; background: var(--box);
      page-break-inside: avoid; margin-bottom: 0.1in;
    }}
    ul {{ margin: 0.04in 0 0.08in; padding-left: 0.2in; }}
    li {{ margin: 0.03in 0; }}
    code {{ font-family: "IBM Plex Mono", Consolas, "Courier New", monospace; font-size: 8.8pt; }}
    pre.source {{
      font-family: "IBM Plex Mono", Consolas, "Courier New", monospace;
      font-size: 7.15pt; line-height: 1.28; white-space: pre-wrap; word-break: break-word;
      background: var(--box); border: 1pt solid var(--line); padding: 0.09in 0.1in;
      margin: 0 0 0.08in;
    }}
    pre.source .ln {{ color: #888; margin-right: 0.12in; user-select: none; }}
    .file {{ page-break-before: always; }}
.footer {{ margin-top: 0.18in; padding-top: 0.08in; border-top: 1pt solid #999; font-size: 8.2pt; }}
    @media print {{
      .no-print {{ display: none !important; }}
      .page {{ max-width: none; padding: 0; }}
      a {{ color: inherit; text-decoration: none; }}
      h2, table, .card {{ break-inside: avoid; }}
      pre.source {{ font-size: 7pt; }}
    }}
    @page {{ size: Letter portrait; margin: 0.45in; }}
  </style>
</head>
<body>
  <div class="toolbar no-print">
    <button type="button" onclick="window.print()">Print / Save as PDF</button>
    <span>Agentic Python source · Letter portrait · {total_lines} lines · {len(CATALOG)} files</span>
  </div>
  <div class="page">
    <p class="kicker">Jarrod Besner · Agentic ····2907</p>
    <h1>Python source book</h1>
    <div class="rule"></div>
<p class="meta">Chat agent <strong>F</strong> and autonomous bot <strong>H</strong> share this
pipeline.
    Generated {html.escape(now)}. Companion file: <code>docs/agentic_python_source_book.py</code>.</p>
    <div class="badge-row">
      <span class="badge">{len(CATALOG)} Python files</span>
      <span class="badge">{total_lines} lines</span>
      <span class="badge">Language: Python 3</span>
      <span class="badge">Does not place orders</span>
    </div>

    <h3>Who uses which code</h3>
    <div class="card">
      <ul>
<li><strong>Agent F (this Cursor chat)</strong> · supervised. Reads the pipeline, reviews tickets via
<code>pipeline/execution.py</code>, and only calls Robinhood <code>place_*</code> after an explicit
            confirm of a specific order. Blocked during RTH while H is enabled.</li>
<li><strong>Agent H (Agentic AI Bot)</strong> · unsupervised Cursor Automation. The standing prompt is
```

```
<code>playbooks/agent_h_autonomous.PROMPT.md</code> (markdown, not Python). On each fire H checks out
            <code>main</code>, reads <code>config/rules.json</code> and the playbooks, and may call
<code>pipeline.bars</code> / <code>pipeline.charts</code> / <code>pipeline.patterns</code> for graphs
and bias. Live <code>place_*</code> is Robinhood MCP from that prompt, not a pipeline side effect.</li>
<li><strong>Not in this book:</strong> lock JSON, playbooks, and the H prompt. Those are not
Python.</li>
      </ul>
    </div>

    <h3>Contents</h3>
    <table>
      <thead><tr><th>Part</th><th>File</th><th>Used by</th><th>Lines</th><th>Role</th></tr></thead>
      <tbody>
        {''.join(toc_rows)}
      </tbody>
    </table>
<p class="note">Each file starts on a new printed page. Source is verbatim Python from the repo.</p>
    {''.join(sections)}
<p class="footer">Agentic Python source book · {html.escape(now)} · {total_lines} lines in
{len(CATALOG)} files ·
    print from this HTML or open <code>docs/agentic_python_source_book.py</code>.</p>
  </div>
</body>
</html>
"""


def build_pdf(path: Path) -> None:
    import pymupdf

    page_w, page_h = 612, 792
    margin = 36
    body_font = 7.2
    header_font = 9.5
    cover_title = 22
    line_h = 9.4
    usable_h = page_h - margin * 2 - 18
    max_lines = int(usable_h / line_h)
    max_chars = 108

    doc = pymupdf.open()
    now = datetime.now(timezone.utc).replace(microsecond=0).strftime("%Y-%m-%d %H:%M UTC")

    def new_page():
        return doc.new_page(width=page_w, height=page_h)

    def footer(page, label: str) -> None:
        page.insert_text(
            pymupdf.Point(margin, page_h - 18),
            f"Agentic Python source  ·  {label}  ·  {now}  ·  page {page.number + 1}",
            fontname="helv",
            fontsize=7,
            color=(0.25, 0.25, 0.25),
        )

    def wrap(text: str, width: int) -> list[str]:
        if len(text) <= width:
            return [text]
        words = text.replace("\t", "    ").split(" ")
        lines: list[str] = []
        cur = ""
        for word in words:
            trial = word if not cur else f"{cur} {word}"
            if len(trial) <= width:
                cur = trial
                continue
            if cur:
                lines.append(cur)
            while len(word) > width:
                lines.append(word[:width])
                word = word[width:]
            cur = word
        if cur:
            lines.append(cur)
```

```
        return lines or [""]

    # Cover
    page = new_page()
    y = 72
page.insert_text(pymupdf.Point(margin, y), "JARROD BESNER  ·  AGENTIC", fontname="helv", fontsize=10)
    y += 36
page.insert_text(pymupdf.Point(margin, y), "Python source book", fontname="helv", fontsize=cover_title)
    y += 28
    page.draw_rect(pymupdf.Rect(margin, y, margin + 90, y + 3), fill=(0, 0, 0), width=0)
    y += 28
    cover_lines = [
        "All Python used by Agent F (supervised Cursor chat) and Agent H",
        "(autonomous Agentic bot). Live place_* is Robinhood MCP, not a",
        "side effect of this code. H's standing prompt is markdown:",
        "playbooks/agent_h_autonomous.PROMPT.md · not included here.",
        "",
        f"{len(CATALOG)} files  ·  generated {now}",
        "Language: Python 3  ·  Letter portrait",
    ]
    for line in cover_lines:
        page.insert_text(pymupdf.Point(margin, y), line, fontname="helv", fontsize=11)
        y += 16
    y += 10
    page.insert_text(pymupdf.Point(margin, y), "Contents", fontname="helv", fontsize=13)
    y += 18
    for rel, part, used_by, role in CATALOG:
        n = _read(rel).count("\n") + 1
        row = f"{part:16s}  {rel:42s}  {used_by:10s}  {n:4d}  {role}"
        for w in wrap(row, 98):
            if y > page_h - 48:
                footer(page, "cover")
                page = new_page()
                y = margin + 12
            page.insert_text(pymupdf.Point(margin, y), w, fontname="cour", fontsize=7)
            y += 10
    footer(page, "cover")

    for rel, part, used_by, role in CATALOG:
        src_lines = _read(rel).splitlines()
        page = new_page()
        header = f"{rel}   ·   {part}   ·   {used_by}   ·   {role}"
page.insert_text(pymupdf.Point(margin, margin + 4), header[:110], fontname="helv",
fontsize=header_font)
        y = margin + 18
page.draw_line(pymupdf.Point(margin, y), pymupdf.Point(page_w - margin, y), color=(0, 0, 0), width=0.8)
        y += 12
        used = 0
        width = max(3, len(str(len(src_lines))))
        for i, raw in enumerate(src_lines, 1):
            prefix = f"{i:{width}d}  "
            wrapped = wrap(raw.replace("\t", "    "), max_chars - len(prefix)) or [""]
            for j, chunk in enumerate(wrapped):
                if used >= max_lines:
                    footer(page, rel)
                    page = new_page()
                    page.insert_text(
                        pymupdf.Point(margin, margin + 4),
                        f"{rel}  (continued)",
                        fontname="helv",
                        fontsize=header_font,
                    )
                    y = margin + 18
                    page.draw_line(
                        pymupdf.Point(margin, y),
                        pymupdf.Point(page_w - margin, y),
                        color=(0, 0, 0),
                        width=0.8,
                    )
                    y += 12
                    used = 0
                line = prefix + chunk if j == 0 else (" " * len(prefix)) + chunk
                page.insert_text(pymupdf.Point(margin, y), line, fontname="cour", fontsize=body_font)
                y += line_h
```

```
                used += 1
        footer(page, rel)

    path.parent.mkdir(parents=True, exist_ok=True)
    doc.save(path)
    doc.close()


def main() -> int:
    DOCS.mkdir(parents=True, exist_ok=True)
    py_book = build_python_book()
    html_doc = build_html(py_book)
    py_path = DOCS / "agentic_python_source_book.py"
    html_path = DOCS / "agentic-python-source-printable.html"
    py_path.write_text(py_book, encoding="utf-8")
    html_path.write_text(html_doc, encoding="utf-8")
    print(f"wrote {py_path} ({py_path.stat().st_size} bytes)")
    print(f"wrote {html_path} ({html_path.stat().st_size} bytes)")
    pdf_candidates = [Path("/dev/shm/agentic-python-source.pdf"), DOCS / "agentic-python-source.pdf"]
    pdf_path = pdf_candidates[0]
    try:
        build_pdf(pdf_path)
        print(f"wrote {pdf_path} ({pdf_path.stat().st_size} bytes)")
        dest = DOCS / "agentic-python-source.pdf"
        if pdf_path != dest:
            dest.write_bytes(pdf_path.read_bytes())
            print(f"copied {dest} ({dest.stat().st_size} bytes)")
    except OSError as exc:
        print(f"pdf write skipped: {exc}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

```python
from pipeline.greeks import delta_in_band, extract_greeks
from pipeline.options_structure import filter_expirations, pick_atm_or_one_otm
from pipeline.patterns import detect_patterns
from pipeline.risk import equity_risk_plan, options_risk_plan
from pipeline.universe import apply_liquidity_filter, extract_watchlist_symbols, option_quote_liquid


def test_extract_excludes_crypto():
    watchlists = [{"id": "1", "display_name": "My First List"}]
    items = {
        "1": [
            {"object_type": "instrument", "symbol": "AAPL"},
            {"object_type": "currency_pair", "symbol": "BTC-USD"},
        ]
    }
    out = extract_watchlist_symbols(watchlists, items, [], include_crypto=False)
    assert out["equity_symbols"] == ["AAPL"]
    assert "BTC-USD" in out["skipped_crypto"]


def test_liquidity_volume_gate():
    fund = {"AAA": {"average_volume": 3_000_000}, "BBB": {"average_volume": 1000}}
    out = apply_liquidity_filter(["AAA", "BBB"], fund, min_average_volume=2_000_000)
    assert out["passed_symbols"] == ["AAA"]
    assert out["rejected"][0]["symbol"] == "BBB"


def test_option_spread_gate():
    preferred, reason, pref_m = option_quote_liquid(
        {"bid_price": "1.00", "ask_price": "1.05"},
        max_spread_pct_of_price=0.1,
        preferred_spread_pct_of_price=0.05,
    )
    assert preferred and reason is None
    assert pref_m["spread_quality"] == "preferred"

    acceptable, reason_ok, acc_m = option_quote_liquid(
        {"bid_price": "1.00", "ask_price": "1.08"},
        max_spread_pct_of_price=0.1,
        preferred_spread_pct_of_price=0.05,
    )
    assert acceptable and reason_ok is None
    assert acc_m["spread_quality"] == "acceptable"

    bad, reason2, _ = option_quote_liquid(
        {"bid_price": "1.00", "ask_price": "1.50"},
        max_spread_pct_of_price=0.1,
        preferred_spread_pct_of_price=0.05,
    )
    assert not bad and reason2 == "spread_too_wide"

    # $0.10 absolute on a cheap contract is ~22% of mid · reject (no dollar override).
    cheap, cheap_reason, cheap_m = option_quote_liquid(
        {"bid_price": "0.40", "ask_price": "0.50"},
        max_spread_pct_of_price=0.1,
        preferred_spread_pct_of_price=0.05,
    )
    assert not cheap and cheap_reason == "spread_too_wide"
    assert cheap_m["spread_pct_of_price"] > 0.1


def test_double_bottom_detection():
    # Equal troughs separated by a bounce; pad so extrema order=3 works.
    prices = (
        [6, 6, 6, 5, 4, 3, 4, 5, 6, 5, 4, 3, 4, 5, 6]
        + [6.2 + i * 0.05 for i in range(20)]
    )
    bars = [{"close": c, "high": c + 0.2, "low": c - 0.2} for c in prices]
    hits = detect_patterns(bars, timeframe="day")
    names = {h["pattern"] for h in hits}
    assert "double_bottom" in names


def test_greeks_no_invention():
```

```
    q = {"delta": "0.45", "gamma": "0.01"}
    pack = extract_greeks(q)
    assert pack["greeks"]["delta"] == 0.45
    assert "theta" in pack["missing_fields"]
    ok, _ = delta_in_band(0.45, lo=0.4, hi=0.5)
    assert ok
    bad, reason = delta_in_band(0.9, lo=0.4, hi=0.5)
    assert not bad and reason is not None


def test_options_risk_math():
    plan = options_risk_plan(premium_per_share=2.0, contracts=1)
    assert plan["cash_risked"] == 200.0
    assert plan["take_profit_pct"] == 0.40
    assert plan["stop_loss_pct"] == 0.20
    assert plan["take_profit_value"] == 280.0
    assert plan["stop_loss_value"] == 160.0
    assert plan["reward_to_risk"] == 2.0
    assert plan["meets_target_rr"] is True


def test_options_risk_bands():
    wide = options_risk_plan(premium_per_share=1.0, stop_loss_pct=0.50, take_profit_pct=1.00)
    assert wide["stop_loss_value"] == 50.0
    assert wide["take_profit_value"] == 200.0
    assert wide["reward_to_risk"] == 2.0
    try:
        options_risk_plan(premium_per_share=1.0, stop_loss_pct=0.07, take_profit_pct=0.50)
        raise AssertionError("expected ValueError for SL below 20%")
    except ValueError as exc:
        assert "stop_loss_pct" in str(exc)
    try:
        options_risk_plan(premium_per_share=1.0, stop_loss_pct=0.25, take_profit_pct=0.20)
        raise AssertionError("expected ValueError for TP below 30%")
    except ValueError as exc:
        assert "take_profit_pct" in str(exc)


def test_equity_risk_math():
    plan = equity_risk_plan(cost_basis=1000.0)
    assert plan["take_profit_value"] == 1250.0
    assert plan["stop_loss_value"] == 800.0


def test_atm_pick_and_dte():
    instruments = [
        {"id": "1", "type": "call", "strike_price": "100"},
        {"id": "2", "type": "call", "strike_price": "105"},
        {"id": "3", "type": "put", "strike_price": "95"},
    ]
    pick = pick_atm_or_one_otm(100.0, instruments, option_type="call")
    assert pick["selection"] == "atm"
    assert pick["instrument"]["id"] == "1"
exps = filter_expirations(["2099-01-01", "2026-09-02"], max_dte=7,
as_of=__import__("datetime").date(2026, 8, 30))
    assert exps == ["2026-09-02"]
```

```python
from pipeline.equity_day_trade import INVERSE_ETF_SYMBOLS
from pipeline.orders import (
    EQUITY_WORKING_STATES,
    OPTION_WORKING_STATES,
    has_working_orders,
    working_orders,
)
from pipeline.io_util import load_rules


def test_working_option_and_equity_states():
    rows = working_orders(
        option_orders=[
            {"id": "o1", "state": "queued"},
            {"id": "o2", "state": "filled"},
            {"id": "o3", "status": "pending_cancelled"},
        ],
        equity_orders=[
            {"id": "e1", "state": "new"},
            {"id": "e2", "state": "cancelled"},
            {"id": "e3", "state": "unconfirmed"},
        ],
    )
    ids = {row["id"] for row in rows}
    assert ids == {"o1", "o3", "e1", "e3"}
    assert has_working_orders([{"state": "confirmed"}], [])
    assert not has_working_orders([{"state": "rejected"}], [{"state": "filled"}])


def test_rules_json_matches_working_states_and_intraday_graphs():
    rules = load_rules()
    assert set(rules["orders"]["option_working_states"]) == set(OPTION_WORKING_STATES)
    assert set(rules["orders"]["equity_working_states"]) == set(EQUITY_WORKING_STATES)
    assert rules["patterns"]["timeframes"] == ["minute", "3minute", "5minute", "hour", "day"]
    assert rules["historicals"]["intraday_interval"] == "minute"
    assert rules["historicals"]["live"] == "get_equity_quotes"
    assert "15minute" not in rules["patterns"]["timeframes"]
    assert "3minute" not in rules["historicals"]["rh_native_intervals"]
    assert rules["options"]["may_hold_overnight_with_stop"] is True
    assert rules["options"]["flatten_at_close"] is False
    assert rules["options"]["overnight_lock_confirmed"] == "2026-08-31"
    assert rules["loop"]["flatten_equity_before_close"] is True
    assert rules["loop"]["options_may_hold_overnight_with_stop"] is True
    assert "flatten_before_close" not in rules["loop"]
    assert rules["git"]["work_on"] == "main"
    assert rules["git"]["open_pull_request"] is False
    assert rules["git"]["create_feature_branch"] is False


def test_permissions_is_kill_switch_not_a_rules_copy():
    import json
    from pathlib import Path

perms = json.loads((Path(__file__).resolve().parents[2] / "config" /
"autonomous_permissions.json").read_text())
    assert perms["status"] == "ACTIVE"
    assert perms["rules_path"] == "config/rules.json"
    assert "universe" not in perms
    assert "risk" not in perms
    assert "patterns" not in perms


def test_inverse_etf_denylist_has_no_duplicate_twm():
    from pathlib import Path

    src = Path(__file__).resolve().parents[1] / "equity_day_trade.py"
    assert src.read_text(encoding="utf-8").count('"TWM"') == 1
    assert "TWM" in INVERSE_ETF_SYMBOLS
```

```python
from pipeline.bars import aggregate_to_minutes, bars_for_timeframe, extract_rh_historicals_bars, normalize_bars
from pipeline.charts import ascii_chart
from pipeline.io_util import load_rules


def _bar(ts: str, o: float, h: float, l: float, c: float, v: float = 10) -> dict:
    return {
        "begins_at": ts,
        "open_price": str(o),
        "high_price": str(h),
        "low_price": str(l),
        "close_price": str(c),
        "volume": v,
        "interpolated": False,
    }


def test_normalize_rh_keys_and_drop_interpolated():
    bars = normalize_bars(
        [
            _bar("2026-09-01T13:30:00Z", 1, 2, 0.5, 1.5, 100),
            {
                "begins_at": "2026-09-01T13:31:00Z",
                "open_price": "1",
                "high_price": "1",
                "low_price": "1",
                "close_price": "1",
                "volume": 1,
                "interpolated": True,
            },
        ]
    )
    assert len(bars) == 1
    assert bars[0]["open"] == 1.0
    assert bars[0]["close"] == 1.5


def test_aggregate_three_minute_from_one_minute():
    # AVGO 2026-09-01 first three RTH 1-minute bars (live RH dump).
    minute = [
        _bar("2026-09-01T13:30:00Z", 364.49, 365.10, 363.36, 364.015, 369411),
        _bar("2026-09-01T13:31:00Z", 364.15, 364.21, 362.55, 362.7262, 41993),
        _bar("2026-09-01T13:32:00Z", 362.56, 363.7399, 362.265, 363.49, 46068),
        _bar("2026-09-01T13:33:00Z", 363.55, 364.50, 362.90, 364.04, 26954),
        _bar("2026-09-01T13:34:00Z", 363.9143, 364.3259, 363.37, 363.37, 18451),
        _bar("2026-09-01T13:35:00Z", 363.35, 364.2381, 363.25, 363.69, 33091),
    ]
    three = aggregate_to_minutes(minute, 3)
    assert [b["begins_at"] for b in three] == [
        "2026-09-01T13:30:00Z",
        "2026-09-01T13:33:00Z",
    ]
    first = three[0]
    assert first["open"] == 364.49
    assert first["high"] == 365.10
    assert first["low"] == 362.265
    assert first["close"] == 363.49
    assert first["volume"] == 369411 + 41993 + 46068


def test_aggregate_five_minute_matches_live_rh_first_bar():
    # Same AVGO session; native RH 5-minute open bar was O 364.49 H 365.10 L 362.265 C 363.37 V 502877.
    minute = [
        _bar("2026-09-01T13:30:00Z", 364.49, 365.10, 363.36, 364.015, 369411),
        _bar("2026-09-01T13:31:00Z", 364.15, 364.21, 362.55, 362.7262, 41993),
        _bar("2026-09-01T13:32:00Z", 362.56, 363.7399, 362.265, 363.49, 46068),
        _bar("2026-09-01T13:33:00Z", 363.55, 364.50, 362.90, 364.04, 26954),
        _bar("2026-09-01T13:34:00Z", 363.9143, 364.3259, 363.37, 363.37, 18451),
    ]
    five = aggregate_to_minutes(minute, 5)
    assert five[0]["begins_at"] == "2026-09-01T13:30:00Z"
    assert five[0]["open"] == 364.49
    assert five[0]["high"] == 365.10
```

```
    assert five[0]["low"] == 362.265
    assert five[0]["close"] == 363.37
    assert five[0]["volume"] == 502877


def test_bars_for_timeframe_synthesizes_3minute():
    minute = [
        _bar("2026-09-01T13:30:00Z", 10, 11, 9, 10.5, 1),
        _bar("2026-09-01T13:31:00Z", 10.5, 10.6, 10.4, 10.4, 1),
        _bar("2026-09-01T13:32:00Z", 10.4, 10.8, 10.3, 10.7, 1),
    ]
    out = bars_for_timeframe({"minute": minute}, "3minute")
    assert len(out) == 1
    assert out[0]["close"] == 10.7


def test_extract_rh_historicals_unwraps_results():
    payload = {
        "data": {
            "results": [
                {
                    "symbol": "AVGO",
                    "interval": "minute",
                    "bars": [_bar("2026-09-01T13:30:00Z", 1, 2, 1, 1.5, 9)],
                }
            ]
        }
    }
    bars = extract_rh_historicals_bars(payload)
    assert len(bars) == 1
    assert bars[0]["close_price"] == "1.5"


def test_ascii_chart_contains_title_and_last_close():
    bars = [
{"begins_at": f"2026-09-01T13:{i:02d}:00Z", "open": 10 + i * 0.1, "high": 11, "low": 9, "close": 10.2 +
i * 0.1, "volume": 1}
        for i in range(12)
    ]
    text = ascii_chart(bars, title="AVGO 1m")
    assert text.startswith("AVGO 1m")
    assert "last O=" in text
    assert "C=11.30" in text


def test_rules_use_live_1m_3m_5m():
    rules = load_rules()
    assert rules["patterns"]["timeframes"] == ["minute", "3minute", "5minute", "hour", "day"]
    hist = rules["historicals"]
    assert hist["live"] == "get_equity_quotes"
    assert hist["intraday_interval"] == "minute"
    assert hist["synthetic_intervals"]["3minute"]["source"] == "minute"
    assert "15minute" not in rules["patterns"]["timeframes"]
    assert "3minute" not in hist["rh_native_intervals"]

# hash-pad 1
```

```python
from datetime import datetime
from zoneinfo import ZoneInfo

from pipeline.equity_day_trade import (
    equity_quote_ok,
    is_inverse_etf,
    regular_hours_buy_ok,
    select_equity_day_trade_candidates,
    whole_share_size,
)
from pipeline.execution import build_equity_entry_proposal, can_place_live
from pipeline.risk import equity_risk_plan
from pipeline.session import ET, entries_open, flatten_window, is_rth, session_gate


def test_session_rth_and_1545_cutoff():
    sunday_noon = datetime(2026, 8, 30, 12, 0, tzinfo=ET)
    monday_open = datetime(2026, 8, 31, 9, 30, tzinfo=ET)
    monday_morning = datetime(2026, 8, 31, 10, 0, tzinfo=ET)
    monday_1545 = datetime(2026, 8, 31, 15, 45, tzinfo=ET)
    monday_close = datetime(2026, 8, 31, 16, 0, tzinfo=ET)
    monday_pre = datetime(2026, 8, 31, 9, 29, tzinfo=ET)

    assert not is_rth(sunday_noon)
    assert is_rth(monday_open)
    assert is_rth(monday_morning)
    assert entries_open(monday_morning)
    assert is_rth(monday_1545)
    assert not entries_open(monday_1545)
    assert flatten_window(monday_1545)
    assert not is_rth(monday_close)
    assert not is_rth(monday_pre)
    gate = session_gate(monday_1545)
    assert gate["reason"] == "no_new_entries_after_1545"


def test_whole_share_size_uses_floor_and_buying_power_cap():
    sized = whole_share_size(1500.0, 347.5)
    assert sized["ok"] is True
    assert sized["shares"] == 4
    assert sized["notional"] == 1390.0
    unaffordable = whole_share_size(1500.0, 1975.0)
    assert unaffordable["ok"] is False
    assert unaffordable["reason"] == "cannot_afford_one_share"
    assert whole_share_size(0, 10)["ok"] is False


def test_no_shorting_skips_bearish_and_inverse_etfs():
    assert is_inverse_etf("SQQQ") is True
    assert is_inverse_etf("SPY") is False
    assert is_inverse_etf("XYZ", {"description": "ProShares UltraShort Inverse"}) is True

    cands, rejected = select_equity_day_trade_candidates(
        symbols=["SQQQ", "AMD", "AAPL"],
        technicals_by_symbol={
            "SQQQ": {"dominant_bias": "bullish"},
            "AMD": {"dominant_bias": "bearish"},
            "AAPL": {"dominant_bias": "bullish"},
        },
        option_candidate_symbols=set(),
        quotes_by_symbol={
            "AAPL": {"bid_price": "100", "ask_price": "100.10"},
            "AMD": {"bid_price": "50", "ask_price": "50.05"},
            "SQQQ": {"bid_price": "10", "ask_price": "10.02"},
        },
        tradability_by_symbol={
            "AAPL": {"regular_hours": {"buy": True}},
            "AMD": {"regular_hours": {"buy": True}},
            "SQQQ": {"regular_hours": {"buy": True}},
        },
        fundamentals_by_symbol={},
        buying_power=1500.0,
        playbook_status="RELEASED",
        take_profit_pct=0.25,
```

```
        stop_loss_pct=0.2,
    )
    reasons = {row["symbol"]: row["reason"] for row in rejected}
    assert reasons["SQQQ"] == "inverse_etf"
    assert reasons["AMD"] == "equity_long_only_requires_bullish"
    assert [c["symbol"] for c in cands] == ["AAPL"]
    assert cands[0]["side"] == "buy"
    assert cands[0]["quantity"] == 14  # floor(1500 / 100.10)


def test_options_priority_and_quote_gates():
    cands, rejected = select_equity_day_trade_candidates(
        symbols=["GOOGL", "META"],
        technicals_by_symbol={
            "GOOGL": {"dominant_bias": "bullish"},
            "META": {"dominant_bias": "bullish"},
        },
        option_candidate_symbols={"GOOGL"},
        quotes_by_symbol={"META": {"bid_price": "0", "ask_price": "350"}},
        tradability_by_symbol={
            "GOOGL": {"regular_hours": {"buy": True}},
            "META": {"regular_hours": {"buy": True}},
        },
        fundamentals_by_symbol={},
        buying_power=1500.0,
        playbook_status="RELEASED",
        take_profit_pct=0.25,
        stop_loss_pct=0.2,
    )
    reasons = {row["symbol"]: row["reason"] for row in rejected}
    assert reasons["GOOGL"] == "options_priority"
    assert reasons["META"] == "one_sided_or_missing_quote"
    assert cands == []


def test_equity_quote_and_tradability_parsers():
    ok, reason, metrics = equity_quote_ok({"bid_price": "10", "ask_price": "10.05"})
    assert ok and reason is None and metrics["ask"] == 10.05
    bad, bad_reason, _ = equity_quote_ok({"bid_price": "10", "ask_price": "0"})
    assert not bad and bad_reason == "one_sided_or_missing_quote"
    assert regular_hours_buy_ok({"regular_hours": {"buy": True}}) == (True, None)
    assert regular_hours_buy_ok(None)[0] is False
    assert regular_hours_buy_ok({"halted": True})[0] is False


def test_equity_risk_prices_from_limit():
    plan = equity_risk_plan(cost_basis=1000.0, shares=10, limit_price=100.0)
    assert plan["take_profit_value"] == 1250.0
    assert plan["stop_loss_value"] == 800.0
    assert plan["stop_price"] == 80.0
    assert plan["take_profit_price"] == 125.0
    assert plan["side"] == "long_only"


def test_equity_place_gate_and_proposal_is_buy_only():
    sunday = datetime(2026, 8, 30, 12, 0, tzinfo=ZoneInfo("America/New_York"))
    monday = datetime(2026, 8, 31, 10, 0, tzinfo=ZoneInfo("America/New_York"))
    ok, reason = can_place_live(
        explicit_confirm=True,
        playbook_released=False,
        playbook_kind="equity",
        h_enabled=False,
        now=sunday,
    )
    assert not ok and reason == "equity_playbook_still_draft"
    ok2, reason2 = can_place_live(
        explicit_confirm=False,
        playbook_released=True,
        playbook_kind="equity",
        h_enabled=False,
        now=sunday,
    )
    assert not ok2 and reason2 == "missing_explicit_user_confirm"
    ok3, reason3 = can_place_live(
```

```
        explicit_confirm=True,
        playbook_released=True,
        playbook_kind="equity",
        h_enabled=False,
        now=sunday,
    )
    assert ok3 and reason3 is None
    blocked, blocked_reason = can_place_live(
        explicit_confirm=True,
        playbook_released=True,
        playbook_kind="equity",
        h_enabled=True,
        now=monday,
    )
    assert not blocked and blocked_reason == "h_owns_rth_while_enabled"

    proposal = build_equity_entry_proposal(
        {
            "symbol": "AAPL",
            "quantity": 4,
            "limit_price": 100.0,
            "playbook_status": "RELEASED",
            "risk": {"stop_loss_pct": 0.2, "take_profit_pct": 0.25},
        }
    )
    assert proposal["side"] == "buy"
    assert proposal["action"] == "buy_to_open"
    assert proposal["quantity"] == "4"
    assert proposal["places_order"] is False
    assert proposal["market_hours"] == "regular_hours"
    assert proposal["stop_price_after_fill"] == 80.0
    assert proposal["take_profit_price"] == 125.0


def test_pipeline_writes_equity_candidate_without_placing(tmp_path, monkeypatch):
    import json

    import pipeline.orchestrator as orch

    signals = tmp_path / "signals"
    journal = tmp_path / "journal"
    signals.mkdir()
    journal.mkdir()
    monkeypatch.setattr(orch, "SIGNALS", signals)
    monkeypatch.setattr(orch, "JOURNAL", journal)

    prices = [6, 6, 6, 5, 4, 3, 4, 5, 6, 5, 4, 3, 4, 5, 6] + [6.2 + i * 0.05 for i in range(20)]
    bars = [{"close": c, "high": c + 0.2, "low": c - 0.2} for c in prices]
    raw = {
        "watchlists": [{"id": "1", "display_name": "T"}],
        "watchlist_items_by_id": {"1": [{"object_type": "instrument", "symbol": "AAPL"}]},
        "fundamentals_by_symbol": {"AAPL": {"average_volume": 3_000_000}},
        "historicals_by_symbol_timeframe": {"AAPL": {"day": bars}},
        "buying_power": 1500.0,
        "equity_quotes_by_symbol": {"AAPL": {"bid_price": "100.00", "ask_price": "100.10"}},
        "equity_tradability_by_symbol": {"AAPL": {"regular_hours": {"buy": True}}},
    }
    summary = orch.run_pipeline(raw)
    assert summary["places_orders"] is False
    assert summary["option_candidate_count"] == 0
    assert summary["equity_candidate_count"] == 1
    payload = json.loads((signals / "equity_candidates.json").read_text())
    cand = payload["candidates"][0]
    assert cand["symbol"] == "AAPL"
    assert cand["side"] == "buy"
    assert cand["structure"] == "long_shares"
    assert cand["quantity"] == 14
    assert cand["playbook_status"] == "RELEASED"
```

```python
from datetime import datetime
from zoneinfo import ZoneInfo

from pipeline.execution import (
    build_option_entry_proposal,
    can_place_live,
    load_latest_equity_candidates,
    load_latest_option_candidates,
)

ET = ZoneInfo("America/New_York")
SUNDAY = datetime(2026, 8, 30, 12, 0, tzinfo=ET)
MONDAY_RTH = datetime(2026, 8, 31, 10, 0, tzinfo=ET)


def test_place_blocked_when_playbook_draft():
    ok, reason = can_place_live(
        explicit_confirm=True, playbook_released=False, h_enabled=False, now=SUNDAY
    )
    assert not ok
    assert reason == "options_playbook_still_draft"


def test_place_blocked_without_confirm():
    ok, reason = can_place_live(
        explicit_confirm=False, playbook_released=True, h_enabled=False, now=SUNDAY
    )
    assert not ok
    assert reason == "missing_explicit_user_confirm"


def test_place_allowed_only_with_confirm_and_release_outside_rth():
    ok, reason = can_place_live(
        explicit_confirm=True, playbook_released=True, h_enabled=True, now=SUNDAY
    )
    assert ok and reason is None


def test_place_blocked_while_h_owns_rth():
    ok, reason = can_place_live(
        explicit_confirm=True, playbook_released=True, h_enabled=True, now=MONDAY_RTH
    )
    assert not ok
    assert reason == "h_owns_rth_while_enabled"


def test_place_allowed_during_rth_if_h_disabled():
    ok, reason = can_place_live(
        explicit_confirm=True, playbook_released=True, h_enabled=False, now=MONDAY_RTH
    )
    assert ok and reason is None


def test_place_allowed_during_rth_with_override():
    ok, reason = can_place_live(
        explicit_confirm=True,
        playbook_released=True,
        h_enabled=True,
        now=MONDAY_RTH,
        h_rth_override=True,
    )
    assert ok and reason is None


def test_proposal_is_one_contract_buy_to_open():
    cand = {
        "symbol": "SOFI",
        "structure": "long_put",
        "expiration": "2026-09-04",
        "option_type": "put",
        "strike": "18.0",
        "option_id": "abc",
        "playbook_status": "DRAFT_NOT_RELEASED",
    }
```

```
    p = build_option_entry_proposal(cand, limit_price="0.43")
    assert p["quantity"] == "1"
    assert p["places_order"] is False
    assert p["legs"][0]["side"] == "buy"
    assert p["legs"][0]["position_effect"] == "open"


def test_load_latest_skips_historical_do_not_place(tmp_path, monkeypatch):
    import json

    import pipeline.execution as execution

    monkeypatch.setattr(execution, "SIGNALS", tmp_path)
    (tmp_path / "option_candidates.json").write_text(
        json.dumps(
            {
                "historical": True,
                "do_not_place": True,
                "candidates": [{"symbol": "SOFI"}],
            }
        )
    )
    (tmp_path / "equity_candidates.json").write_text(
        json.dumps({"do_not_place": True, "candidates": [{"symbol": "AAPL"}]})
    )
    assert load_latest_option_candidates() == []
    assert load_latest_equity_candidates() == []

    (tmp_path / "option_candidates.json").write_text(
        json.dumps({"candidates": [{"symbol": "NU"}]})
    )
    assert load_latest_option_candidates()[0]["symbol"] == "NU"
```